



Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего профессионального образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Фундаментальные науки»

КАФЕДРА «Высшая математика»

РАСЧЁТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ НА ТЕМУ:

Моделирование распространения ударной волны
в газе с локальным энерговложением
средствами библиотеки JAX

Студент группы ФН1-81Б

Д.С. Филькин

(Подпись, дата)

Руководитель ВКР

О.В. Кравченко

(Подпись, дата)

Нормоконтролер

А.А. Польшкова

(Подпись, дата)

2026 г.

РЕФЕРАТ

Расчётно-пояснительная записка состоит из 73 страниц, 18 рисунков, 13 таблиц, 17 источников, 4 глав и 1 приложения.

Ключевые слова: ударная волна, локальное энерговложение, уравнения Эйлера, центральные разностные схемы высокого разрешения, схемы SD2 и FD2, библиотека JAX, дифференцируемое программирование, ускорение на GPU, вычислительная газовая динамика.

Целью данной работы является численное моделирование распространения ударной волны в газе с локальным энерговложением средствами разработанного высокопроизводительного программного комплекса на базе библиотеки JAX.

В соответствии с поставленной целью в работе решаются следующие задачи:

1. Программная адаптация и высокопроизводительная реализация семейства центральных разностных схем высокого разрешения SD2 и FD2 на базе библиотеки JAX;
2. Кросс-верификация разработанного комплекса путём сравнения с алгоритмом точного решения задачи Римана, а также с данными открытых CFD-библиотек *centpy* и JAX-FLUIDS;
3. Оценка вычислительной эффективности реализованных алгоритмов и анализ аппаратного ускорения на центральном (CPU) и графическом (GPU) процессорах;
4. Демонстрация возможностей разработанного комплекса на задаче моделирования динамики параметров газа при прохождении ударной волны через зону локального энерговложения.

Содержание

РЕФЕРАТ	2
ВВЕДЕНИЕ	5
1. Математическая модель	7
1.1. Система уравнений Эйлера	7
1.2. Квазилинейная форма уравнений и характеристические скорости	10
1.3. Классическая задача о распаде произвольного разрыва	13
1.4. Постановка модифицированной задачи	15
2. Построение разностных схем	22
2.1. Семейство центральных схем высокого разрешения	22
2.2. Полностью дискретный подход	23
2.2.1. Интегро-интерполяционный метод конечных объемов	23
2.2.2. Алгоритм схемы Нессяху–Тадмора второго порядка	26
2.3. Полудискретный подход	29
2.3.1. Полудискретная форма уравнений	29
2.3.2. Алгоритм схемы Курганова–Тадмора второго порядка с линейной реконструкцией	32
3. Программная реализация	35
3.1. Библиотека для вычислений JAX	35
3.2. Функциональная парадигма JAX	36
3.3. Архитектура программного комплекса	37
4. Численное исследование	39
4.1. Верификация численных методов	39
4.2. Анализ модифицированных задач	49
4.3. Кросс-верификация относительно JAX-FLUIDS	56

4.4. Анализ аппаратного ускорения	66
ЗАКЛЮЧЕНИЕ	72
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	75
ПРИЛОЖЕНИЕ А	77

ВВЕДЕНИЕ

В физике и технике возникают проблемы, связанные с необходимостью изучения высокоскоростных течений и взаимодействия скачков уплотнения с неоднородностями среды. Такие задачи ставят, например, аэродинамика больших скоростей, проектирование сверхзвуковых двигателей и разработка перспективных летательных аппаратов. В связи с этим в настоящее время одной из актуальных областей прикладной газовой динамики является изучение процессов распространения ударных волн в газе с локальным энергосложением, которое открывает новые методы управления аэродинамическими характеристиками и снижения волнового сопротивления [1].

Для описания газодинамической среды идеального газа используется система уравнений Эйлера [2]. Непосредственное аналитическое решение таких систем возможно лишь в ограниченном ряде случаев, например в классической задаче Римана для одномерного случая [3, 4]. Однако подобные точные решения позволяют изучить лишь качественные особенности поведения объекта. В инженерных же приложениях одной из главных задач является получение точных количественных результатов. Поэтому возникает потребность в численных методах для задач газовой динамики.

Более того, для всестороннего анализа подобных физических процессов часто требуется проведение ресурсоемких расчетов, зависящих от различных варьируемых параметров. Большинство современных пакетов для газодинамического моделирования опираются на классические методы программирования для CPU, что при серийных расчетах на мелких сетках требует значительных временных затрат или доступа к дорогостоящим суперкомпьютерам. В связи с этим возникает потребность в разработке эффективного программного комплекса, переносящего известный класс численных методов на современные архитектуры, с использованием инструментов дифференцируемого программирования и аппаратного ускорения. При разработке программного комплекса, реализующего численное моделирование распространения удар-

ной волны, будем опираться на следующие основные принципы:

- использование математической библиотеки высокопроизводительных вычислений JAX [5] для обеспечения векторизации и JIT-компиляции;
- вычисления на GPU с использованием облачной платформы Google Colab для программного ускорения ресурсоемких задач;
- интеграция с открытыми библиотеками Matplotlib/SciencePlots [6] для визуализации

Целью работы является численное моделирование распространения ударной волны в газе с локальным энерговыделением средствами разработанного высокопроизводительного программного комплекса на базе библиотеки JAX. Для достижения этой цели были решены следующие задачи:

1. программная адаптация и высокопроизводительная реализация семейства центральных разностных схем SD2, FD2 [7, 8] на базе библиотеки JAX;
2. проведение кросс-верификации разработанного комплекса путем сравнения с данными открытых CFD-библиотек *centpy* [9] и JAX-FLUIDS [10], а также с алгоритмом точного решения задачи Римана;
3. оценка вычислительной эффективности реализованных алгоритмов и анализ аппаратного ускорения на CPU и GPU;
4. демонстрация возможностей разработанного комплекса на задаче моделирования динамики параметров газа при прохождении ударной волны через зону локального энерговыделения.

1. Математическая модель

1.1. Система уравнений Эйлера

Течение идеального сжимаемого невязкого газа описывается системой дифференциальных уравнений Эйлера [2], которая выражает фундаментальные физические законы: сохранения массы, импульса и полной энергии.

Для произвольного фиксированного контрольного объема Ω с границей $\partial\Omega$ в отсутствие внешних массовых сил и источников тепловыделения эти законы в интегральной форме формулируются следующим образом:

- Закон сохранения массы: изменение массы газа внутри объема равно макроскопическому потоку массы через его границу

$$\frac{\partial}{\partial t} \int_{\Omega} \rho d\Omega + \oint_{\partial\Omega} \rho(\mathbf{v} \cdot \mathbf{n}) dS = 0. \quad (1)$$

- Закон сохранения импульса: изменение импульса внутри объема обусловлено потоком импульса через границу и действием поверхностных сил газодинамического давления p

$$\frac{\partial}{\partial t} \int_{\Omega} \rho \mathbf{v} d\Omega + \oint_{\partial\Omega} \rho \mathbf{v}(\mathbf{v} \cdot \mathbf{n}) dS + \oint_{\partial\Omega} p \mathbf{n} dS = 0. \quad (2)$$

- Закон сохранения полной энергии: изменение полной энергии газа равно работе сил давления и конвективному переносу энергии через границу

$$\frac{\partial}{\partial t} \int_{\Omega} \rho E d\Omega + \oint_{\partial\Omega} (\rho E + p)(\mathbf{v} \cdot \mathbf{n}) dS = 0. \quad (3)$$

В приведенных выражениях ρ — плотность газа, $\mathbf{v}$ — вектор скорости течения, p — давление, E — удельная полная энергия, $\mathbf{n}$ — вектор внешней нормали к элементу поверхности dS .

Применяя к интегральным законам сохранения теорему Остроградского-Гаусса, получаем систему уравнений в дифференциальной форме. Для двумерного течения в декартовой системе координат, где вектор скорости имеет

компоненты $\mathbf{v} = (u, v)^T$, она принимает вид

$$\begin{cases} \frac{\partial \rho}{\partial t} + \frac{\partial(\rho u)}{\partial x} + \frac{\partial(\rho v)}{\partial y} = 0, \\ \frac{\partial(\rho u)}{\partial t} + \frac{\partial(\rho u^2 + p)}{\partial x} + \frac{\partial(\rho uv)}{\partial y} = 0, \\ \frac{\partial(\rho v)}{\partial t} + \frac{\partial(\rho uv)}{\partial x} + \frac{\partial(\rho v^2 + p)}{\partial y} = 0, \\ \frac{\partial(\rho E)}{\partial t} + \frac{\partial(u(\rho E + p))}{\partial x} + \frac{\partial(v(\rho E + p))}{\partial y} = 0. \end{cases} \quad (4)$$

Полученная система (4) записана в консервативной (дивергентной) форме. При моделировании распространения ударных волн использование именно такой формы принципиально важно: только она гарантирует корректное выполнение законов сохранения на поверхностях разрывов и позволяет правильно рассчитывать параметры ударных волн в соответствии с условиями Ренкина-Гюгонио [11, 12].

Для компактности систему (4) удобно представить в матричной (векторной) форме. В двумерном пространственном случае в декартовой системе координат $D(x, y)$ она записывается в виде матричного гиперболического уравнения

$$\frac{\partial U}{\partial t} + \frac{\partial F(U)}{\partial x} + \frac{\partial G(U)}{\partial y} = 0, \quad (5)$$

где U — вектор консервативных переменных, а $F(U)$ и $G(U)$ — векторы физических потоков вдоль осей x и y соответственно. Данные векторы определяются следующим образом:

$$U = \begin{pmatrix} \rho \\ \rho u \\ \rho v \\ \rho E \end{pmatrix}, \quad F(U) = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ u(\rho E + p) \end{pmatrix}, \quad G(U) = \begin{pmatrix} \rho v \\ \rho uv \\ \rho v^2 + p \\ v(\rho E + p) \end{pmatrix}, \quad (6)$$

где:

- ρ — плотность газа [кг/м³];
- u, v — компоненты вектора скорости газа вдоль осей x и y [м/с];
- $\rho u, \rho v$ — компоненты вектора плотности импульса [кг/(м²·с)];
- p — газодинамическое давление [Па];
- E — полная удельная энергия газа, в расчете на единицу массы [Дж/кг], определяемая как

$$E = e + \frac{u^2 + v^2}{2}, \quad (7)$$

где e — удельная внутренняя энергия;

- ρE — плотность полной энергии (энергия в единице объема) [Дж/м³].

Система уравнений (5)–(6) является незамкнутой, так как содержит пять неизвестных величин (ρ, u, v, p, E) при четырех уравнениях. Для замыкания системы используется термодинамическое уравнение состояния. Для идеального газа оно имеет вид

$$p = (\gamma - 1)\rho e, \quad (8)$$

где γ — показатель адиабаты.

Выражая удельную внутреннюю энергию через полную энергию

$$e = E - \frac{u^2 + v^2}{2}, \quad (9)$$

уравнение (8) можно переписать в форме, удобной для прямого вычисления давления из компонент вектора U

$$p = (\gamma - 1) \left(\rho E - \frac{(\rho u)^2 + (\rho v)^2}{2\rho} \right). \quad (10)$$

В случае рассмотрения одномерных течений, градиенты параметров по оси y полагаются равными нулю $\frac{\partial}{\partial y} = 0$, а компонента скорости v и вектор потока $G(U)$ исключаются из системы, что сводит её (5) к одномерному

случаю.

В реальных же приложениях взаимодействие плоского фронта ударной волны с нагретой областью носит существенно многомерный характер. Тем не менее, исследование данной проблемы методически верно начинать с одномерной постановки. Во-первых, одномерное сечение по нормали к зонам нагрева позволяет исключить поперечные потоки и изолированно изучить фундаментальные механизмы рефракции фронта. Во-вторых, применяемые в работе схемы сквозного счета базируются на методе расщепления по направлениям. Двумерный расчет сводится к последовательному решению множества локальных 1D-задач. Таким образом, верифицированная одномерная модель является базовым математическим и алгоритмическим ядром разрабатываемого программного комплекса.

1.2. Квазилинейная форма уравнений и характеристические скорости

Приведем подробный вывод матрицы Якоби и её собственных значений для одномерных уравнений Эйлера [3]. Система уравнений Эйлера в консервативной форме имеет вид

$$\frac{\partial U}{\partial t} + \frac{\partial F(U)}{\partial x} = 0, \quad (11)$$

где U — вектор консервативных переменных, а $F(U)$ — вектор потоков

$$U = \begin{pmatrix} U_1 \\ U_2 \\ U_3 \end{pmatrix} = \begin{pmatrix} \rho \\ \rho u \\ \rho E \end{pmatrix}, \quad F(U) = \begin{pmatrix} F_1 \\ F_2 \\ F_3 \end{pmatrix} = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ u(\rho E + p) \end{pmatrix}. \quad (12)$$

Здесь ρ — плотность, u — скорость, p — давление, E — удельная полная энергия (отнесённая к единице массы), так что ρE — полная энергия единицы объёма.

Для замыкания системы используется уравнение состояния идеального газа

$$p = (\gamma - 1) \left(\rho E - \frac{\rho u^2}{2} \right), \quad (13)$$

где γ — показатель адиабаты. Выразим давление p исключительно через компоненты вектора U

$$p = (\gamma - 1) \left(U_3 - \frac{1}{2} \frac{U_2^2}{U_1} \right). \quad (14)$$

Теперь перепишем компоненты вектора потоков F через переменные U_1, U_2, U_3

$$F_1 = U_2, \quad (15)$$

$$F_2 = \frac{U_2^2}{U_1} + (\gamma - 1) \left(U_3 - \frac{1}{2} \frac{U_2^2}{U_1} \right) = \frac{3 - \gamma}{2} \frac{U_2^2}{U_1} + (\gamma - 1) U_3, \quad (16)$$

$$F_3 = \frac{U_2}{U_1} \left(U_3 + (\gamma - 1) \left(U_3 - \frac{1}{2} \frac{U_2^2}{U_1} \right) \right) = \gamma \frac{U_2 U_3}{U_1} - \frac{\gamma - 1}{2} \frac{U_2^3}{U_1^2}. \quad (17)$$

Для перехода к квазилинейной форме

$$\frac{\partial U}{\partial t} + A(U) \frac{\partial U}{\partial x} = 0 \quad (18)$$

необходимо найти матрицу Якоби $A = \frac{\partial F}{\partial U}$. Вычислим её элементы $A_{ij} = \frac{\partial F_i}{\partial U_j}$ почленно.

Первая строка $A_{11} = 0, \quad A_{12} = 1, \quad A_{13} = 0$.

Вторая строка (возвращаясь к физическим переменным через подстановку $u = U_2/U_1$):

$$A_{21} = -\frac{3 - \gamma}{2} \frac{U_2^2}{U_1^2} = \frac{\gamma - 3}{2} u^2, \quad (19)$$

$$A_{22} = (3 - \gamma) \frac{U_2}{U_1} = (3 - \gamma) u, \quad (20)$$

$$A_{23} = \gamma - 1. \quad (21)$$

Для вывода третьей строки введём полную удельную энтальпию

$$H = \frac{\rho E + p}{\rho}. \quad (22)$$

Из уравнения состояния следует, что

$$\gamma E = H + \frac{\gamma - 1}{2} u^2. \quad (23)$$

Вычисляем производные

$$\begin{aligned} A_{31} &= -\gamma \frac{U_2 U_3}{U_1^2} + (\gamma - 1) \frac{U_2^3}{U_1^3} \\ &= -u (\gamma E - (\gamma - 1) u^2) \\ &= u \left(\frac{\gamma - 1}{2} u^2 - H \right), \end{aligned} \quad (24)$$

$$A_{32} = \gamma \frac{U_3}{U_1} - \frac{3(\gamma - 1)}{2} \frac{U_2^2}{U_1^2} = \gamma E - \frac{3}{2} (\gamma - 1) u^2 = H - (\gamma - 1) u^2, \quad (25)$$

$$A_{33} = \gamma \frac{U_2}{U_1} = \gamma u. \quad (26)$$

Собирая вычисленные элементы вместе, получаем матрицу Якоби A

$$A = \begin{pmatrix} 0 & 1 & 0 \\ \frac{\gamma-3}{2} u^2 & (3 - \gamma) u & \gamma - 1 \\ u \left(\frac{\gamma-1}{2} u^2 - H \right) & H - (\gamma - 1) u^2 & \gamma u \end{pmatrix}. \quad (27)$$

Скорости распространения волн определяются собственными значениями λ матрицы A , которые являются корнями характеристического уравнения

$$\det(A - \lambda I) = 0. \quad (28)$$

Раскрытие данного определителя приводит к кубическому уравнению

относительно λ . Вводя скорость звука

$$a = \sqrt{(\gamma - 1) \left(H - \frac{u^2}{2} \right)}, \quad (29)$$

находим три действительных корня характеристического уравнения

$$\lambda_1 = u - a, \quad \lambda_2 = u, \quad \lambda_3 = u + a. \quad (30)$$

Поскольку характеристическое уравнение имеет степень $n = 3$, система обладает тремя действительными собственными значениями, что подтверждает её гиперболичность [4]. Каждому собственному значению отвечает своё семейство характеристик в плоскости (x, t) . Физически это означает, что при распаде произвольного разрыва образуются три волны: две акустические (λ_1 и λ_3), отвечающие нелинейным характеристическим полям (ударные волны или волны разрежения), и одна энтропийная (λ_2), отвечающая линейно вырожденному полю (контактному разрыву). Именно эти скорости в дальнейшем используются для вычисления численной вязкости в центральных схемах Курганова–Тадмора [8].

1.3. Классическая задача о распаде произвольного разрыва

В вычислительной газовой динамике фундаментальную роль играет задача Римана о распаде произвольного разрыва [3, 11]. Математически классическая одномерная задача Римана представляет собой задачу Коши для системы уравнений Эйлера с начальными условиями в виде кусочно-постоянных функций, имеющих единственный разрыв в точке $x = x_0$:

$$U(x, 0) = \begin{cases} U_L, & \text{при } x < x_0 \\ U_R, & \text{при } x > x_0 \end{cases} \quad (31)$$

где U_L и U_R — векторы консервативных параметров слева и справа от разрыва.

Опираясь на полученный ранее характеристический анализ (наличие трех волн $\lambda_1, \lambda_2, \lambda_3$), математическая теория показывает, что распад начального разрыва приводит к формированию автомодельного течения, зависящего только от переменной x/t . Структура этого течения состоит из левой волны (ударной или веера разрежения), правой волны (ударной или веера разрежения) и центрального контактного разрыва, на котором давление p_* и скорость u_* непрерывны, а плотность терпит скачок.

Таким образом, нахождение точного решения сводится к определению параметров газа в так называемой «звездной области» (Star Region), заключенной между левой и правой волнами.

Ключевым этапом алгоритма точного решателя Римана (Exact Riemann Solver), подробно описанного Э. Торо [3], является нахождение давления p_* в контактном разрыве. Для этого выводится единое нелинейное алгебраическое уравнение

$$f(p_*, U_L, U_R) \equiv f_L(p_*, U_L) + f_R(p_*, U_R) + \Delta u = 0, \quad (32)$$

где $\Delta u = u_R - u_L$ — разность начальных скоростей, а функции f_K ($K = L, R$) задают изменение скорости на соответствующей волне в зависимости от того, является ли она волной разрежения (изоэнтропический процесс) или ударной волной (соотношения Ренкина-Гюгонио)

$$f_K(p, U_K) = \begin{cases} (p - p_K) \sqrt{\frac{A_K}{p + B_K}}, & \text{если } p > p_K \text{ ударная волна,} \\ \frac{2a_K}{\gamma - 1} \left[\left(\frac{p}{p_K} \right)^{\frac{\gamma - 1}{2\gamma}} - 1 \right], & \text{если } p \leq p_K \text{ волна разрежения,} \end{cases} \quad (33)$$

здесь $a_K = \sqrt{\gamma p_K / \rho_K}$ — локальная скорость звука, а

$$A_K = \frac{2}{(\gamma + 1)\rho_K}, \quad B_K = \frac{\gamma - 1}{\gamma + 1} p_K \quad (34)$$

- газодинамические константы.

Поскольку функция $f(p_*)$ является монотонно возрастающей и строго выпуклой, нелинейное уравнение (32) эффективно и с любой заданной точностью разрешается итерационным методом Ньютона-Рафсона.

После того как давление p_* найдено, скорость u_* вычисляется аналитически через функции f_L и f_R . Зная параметры в контактном разрыве и типы волн (определяемые соотношением p_* и $p_{L,R}$), плотности ρ_{*L} и ρ_{*R} вычисляются по уравнениям адиабаты или Гюгонио соответственно. Наконец, пространственное распределение всех параметров $\rho(x, t), u(x, t), p(x, t)$ в любой момент времени t восстанавливается по аналитическим формулам траекторий волн (лучей веера Римана или скоростей ударных фронтов).

1.4. Постановка модифицированной задачи

В данной работе рассматривается процесс импульсного (изохорного) локального подвода энергии в газ. Поскольку характерное время выделения энергии существенно меньше характерных газодинамических времен, газ в области энерговложения не успевает расшириться. Это приводит к локальному скачку температуры и давления с последующим газодинамическим выравниванием, в результате которого формируется область с существенно пониженной плотностью газа по сравнению с невозмущенной средой (эффект теплового следа) [1].

Подобная физическая формулировка позволяет отказаться от явного введения источников членов в систему уравнений Эйлера и свести проблему к решению задачи Коши с профилированными начальными условиями. Расчетная область одномерной задачи $x \in [0, 1]$ делится на три характерные

зоны:

- *Зона инициации ударной волны* ($x \in [x_{sw}, 1]$): содержит сжатый газ (U_{sw}), генерирующий падающую ударную волну, которая движется в отрицательном направлении оси x (справа налево).
- *Зона невозмущенного газа*: содержит покоящийся газ с фоновыми параметрами $U_0 = (\rho_0, u_0, p_0)^T$, где $u_0 = 0$.
- *Зона локального энерговложения* ($x \in [x_{e1}, x_{e2}]$): задается через параметр локального минимума плотности

$$\alpha = \frac{\rho_e}{\rho_0} < 1. \quad (35)$$

Внутри этой зоны давление равно фоновому p_0 , а плотность равна $\alpha\rho_0$.

Для корректного моделирования параметров зоны инициации U_{sw} необходимо, чтобы в невозмущенный газ двигалась строго одна изолированная ударная волна с заданным числом Маха M_s в отрицательном направлении оси x . Параметры за фронтом стационарной ударной волны жестко связаны с параметрами невозмущенного газа классическими соотношениями Ренкина-Гюгонио [2], выражающими законы сохранения массы, импульса и энергии на поверхности разрыва

$$\begin{cases} \rho_{sw}(u_{sw} - D) = \rho_0(u_0 - D), \\ \rho_{sw}(u_{sw} - D)^2 + p_{sw} = \rho_0(u_0 - D)^2 + p_0, \\ (u_{sw} - D)(\rho_{sw}E_{sw} + p_{sw}) = (u_0 - D)(\rho_0E_0 + p_0), \end{cases} \quad (36)$$

где $D = -M_s a_0$ — скорость движения фронта ударной волны в лабораторной системе координат (знак «минус» отвечает распространению справа налево), a_0 — скорость звука в невозмущенном газе.

Разрешая систему (36) относительно параметров сжатого газа при усло-

вии $u_0 = 0$, получаем явные формулы для задания вектора U_{sw}

$$\begin{cases} p_{sw} = p_0 \left[1 + \frac{2\gamma}{\gamma + 1} (M_s^2 - 1) \right], \\ \rho_{sw} = \rho_0 \left[\frac{(\gamma + 1)M_s^2}{(\gamma - 1)M_s^2 + 2} \right], \\ u_{sw} = -a_0 \frac{2}{\gamma + 1} \left(M_s - \frac{1}{M_s} \right). \end{cases} \quad (37)$$

Используя полученные параметры (37), вектор начальных условий $U(x, 0)$ для задачи с энерговложением принимает следующий вид

$$(\rho, u, p)|_{t=0} = \begin{cases} (\rho_{sw}, u_{sw}, p_{sw}), & x \in [x_{sw}, 1], \\ (\alpha\rho_0, 0, p_0), & x \in [x_{e1}, x_{e2}], \\ (\rho_0, 0, p_0), & \text{в остальных частях области.} \end{cases} \quad (38)$$

Для проведения численного эксперимента и верификации реализованных алгоритмов зафиксированы физические параметры модельной среды. В качестве рабочего тела рассматривается идеальный двухатомный газ с показателем адиабаты $\gamma = 1.4$. Фоновые параметры невозмущенной среды нормированы: $\rho_0 = 1.0$, $p_0 = 1.0$, $u_0 = 0.0$.

Пусть инициирующая ударная волна имеет число Маха $M_s = 2.0$. Выбор данного режима обусловлен следующими физическими и методологическими соображениями. С одной стороны, при малых числах Маха ($M_s \rightarrow 1$) интенсивность волны падает, она приближается к акустической, из-за чего нелинейные эффекты взаимодействия становятся слабовыраженными. С другой стороны, при высоких сверхзвуковых скоростях ($M_s > 4$) температура за фронтом волны возрастает настолько, что начинают играть существенную роль реальные физико-химические свойства газа, что делает использование модели совершенного газа с постоянным показателем адиабаты $\gamma = 1.4$ физически некорректным. Режим $M_s = 2.0$ представляет собой оптимальный ба-

ланс: он обеспечивает развитие выраженных нелинейных газодинамических структур при строгом соблюдении границ применимости выбранной математической модели.

Подстановка заданных величин в аналитические соотношения Ренкина-Гюгонно (37) позволяет однозначно определить параметры сжатого газа в зоне инициации

$$\begin{cases} p_{sw} = 1.0 \left[1 + \frac{2 \cdot 1.4}{2.4} (4.0 - 1) \right] = 4.5, \\ \rho_{sw} = 1.0 \left[\frac{2.4 \cdot 4.0}{0.4 \cdot 4.0 + 2} \right] \approx 2.667, \\ u_{sw} = -\sqrt{1.4} \cdot \frac{2}{2.4} (2.0 - 0.5) \approx -1.479. \end{cases} \quad (39)$$

Зона локального энерговложения характеризуется параметром плотности $\alpha = 0.1$, что соответствует десятикратному изобарному уменьшению плотности газа. Геометрия расчетной области для одномерного случая определяется следующими пространственными интервалами: $x \in [0.0, 1.0]$ — общая длина расчетного домена; $x \in [0.8, 1.0]$ — зона генерации ударной волны; $x \in [0.3, 0.5]$ — зона теплового следа. Оставшиеся участки заполнены невозмущенным газом.

Поскольку ударная волна иницируется у правой границы и движется справа налево (в отрицательном направлении оси x), вектор скорости за фронтом принимает отрицательное значение ($u_{sw} \approx -1.479$). Вектор начальных макроскопических параметров (38) принимает следующий числовой вид:

$$(\rho, u, p)|_{t=0} = \begin{cases} (1.0, 0.0, 1.0), & \text{при } x \in [0.0, 0.3), \\ (0.1, 0.0, 1.0), & \text{при } x \in [0.3, 0.5], \\ (1.0, 0.0, 1.0), & \text{при } x \in (0.5, 0.8), \\ (2.667, -1.479, 4.5), & \text{при } x \in [0.8, 1.0]. \end{cases} \quad (40)$$

Для корректного замыкания системы на границах расчетной области задаются смешанные граничные условия. На левой границе ($x = 0$) ставится условие Неймана (свободный выход), обеспечивающее беспрепятственное прохождение волн за пределы расчетного домена за счет равенства нулю пространственных градиентов всех макроскопических параметров. На правой границе ($x = 1$) задается физическое условие непротекания (идеальная жесткая стенка), математически выражающееся равенством нулю нормальной компоненты скорости при сохранении нулевого градиента для термодинамических величин:

$$\left. \frac{\partial q}{\partial x} \right|_{x=0} = 0, \quad u|_{x=1} = 0, \quad \left. \frac{\partial \rho}{\partial x} \right|_{x=1} = 0, \quad \left. \frac{\partial p}{\partial x} \right|_{x=1} = 0, \quad (41)$$

где $q = (\rho, u, p)^T$ — вектор макроскопических параметров.

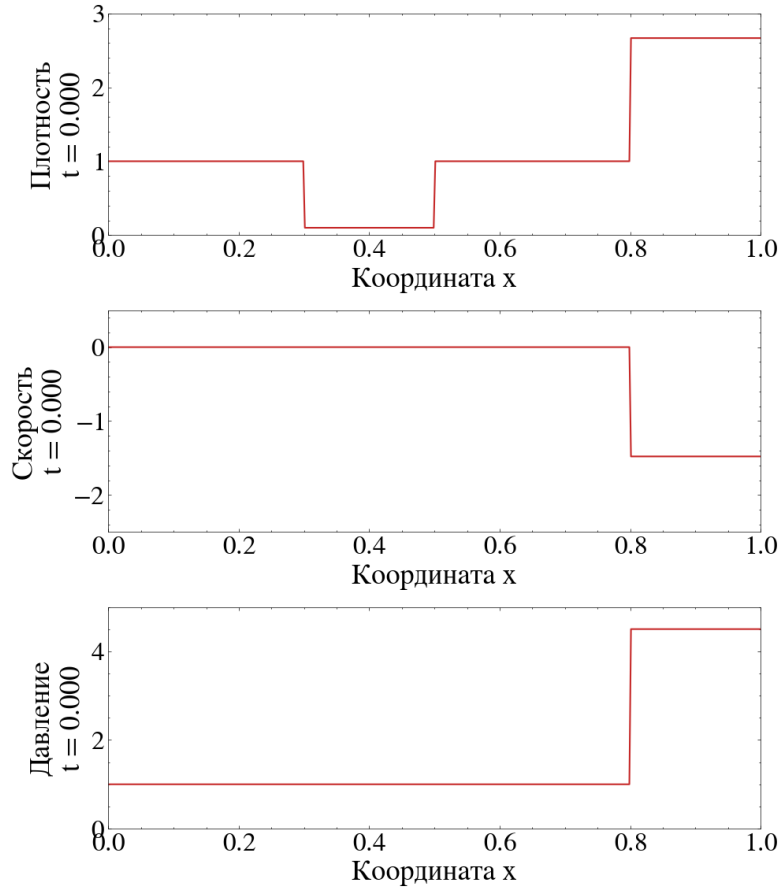


Рисунок 1. Постановка модифицированной задачи в одномерном случае

Двумерная модификация задачи формируется путем расширения одно-

мерной постановки на прямоугольный расчетный домен $(x, y) \in [0, 1] \times [0, 1]$.

Зона инициации ударной волны задается как вертикальная полоса $x \geq 0.8$ для всех $y \in [0, 1]$. Подобная конфигурация генерирует плоский фронт падающей ударной волны, движущийся параллельно оси x справа налево. Газодинамические параметры в данной зоне соответствуют одномерному случаю, при этом поперечная компонента скорости тождественно равна нулю $v_{sw} = 0$.

С целью исследования истинно двумерных эффектов рефракции на различных геометрических формах, зона локального энерговложения состоит из двух изолированных областей пониженной плотности. Первая область представляет собой круг с центром в точке $(x_{c1}, y_{c1}) = (0.45, 0.65)$ и радиусом $R_1 = 0.08$. Вторая область имеет форму квадрата и ограничена координатами $x \in [0.25, 0.41]$, $y \in [0.20, 0.44]$. Вектор начальных условий имеет вид:

$$(\rho, u, v, p)|_{t=0} = \begin{cases} (2.667, -1.479, 0.0, 4.5), & \text{при } x \geq 0.8, \\ (0.1, 0.0, 0.0, 1.0), & \text{внутри зон энерговложений,} \\ (1.0, 0.0, 0.0, 1.0), & \text{в остальных точках домена.} \end{cases} \quad (42)$$

Для корректного замыкания расчетной области применяются смешанные граничные условия. На левой границе ($x = 0$) задается условие Неймана (свободный выход), обеспечивающее нулевой градиент всех макроскопических параметров для беспрепятственного выхода волн. На правой ($x = 1$), нижней ($y = 0$) и верхней ($y = 1$) границах задаются физические условия непротекания (жесткая стенка). Математически это выражается равенством нулю нормальной компоненты скорости при сохранении нулевого градиента для термодинамических величин и тангенциальной скорости:

$$\left. \frac{\partial q}{\partial x} \right|_{x=0} = 0, \quad u|_{x=1} = 0, \quad v|_{y=0,1} = 0, \quad \left. \frac{\partial \rho}{\partial n} \right|_{\Gamma_w} = 0, \quad \left. \frac{\partial p}{\partial n} \right|_{\Gamma_w} = 0, \quad (43)$$

где $\mathbf{q} = (\rho, u, v, p)^T$ — вектор макроскопических параметров, а Γ_w обозначает границы с условием непротекания (правую, верхнюю и нижнюю).

Данная постановка позволяет корректно моделировать процессы нелинейного взаимодействия разрывов и рефракции на скругленной и острой геометриях.

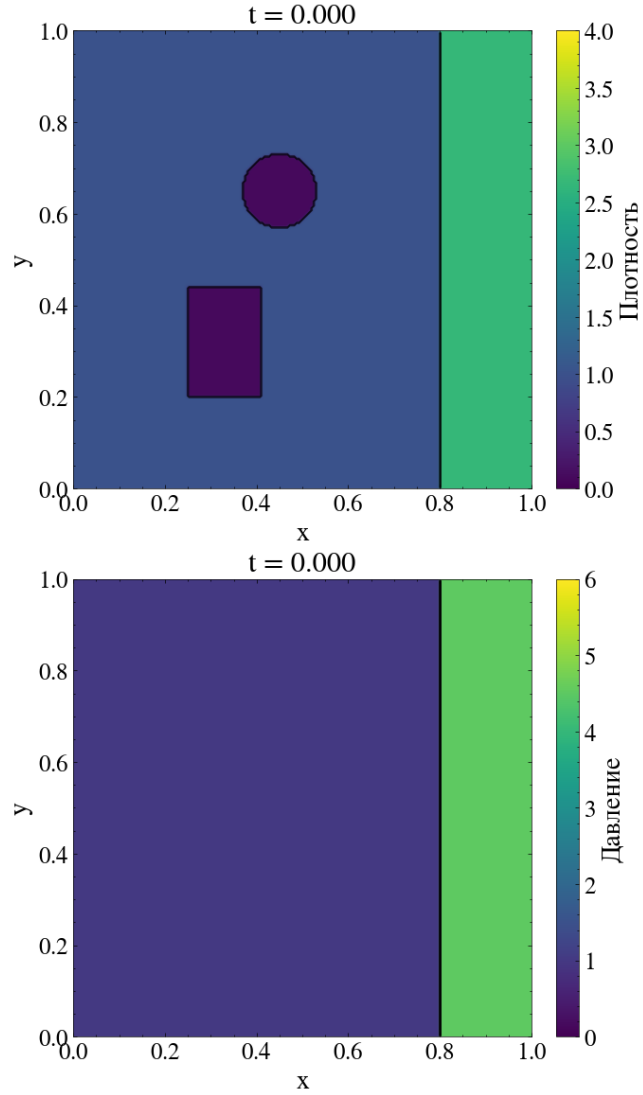


Рисунок 2. Постановка модифицированной задачи в двумерном случае

2. Построение разностных схем

2.1. Семейство центральных схем высокого разрешения

При численном решении гиперболических систем традиционно используются два принципиально разных класса методов: противопотоковые (upwind) схемы, к которым относятся в том числе методы типа Годунова [11], и центральные схемы.

Методы типа Годунова обладают высокой разрешающей способностью на скачках, однако требуют точного или приближенного решения локальной задачи Римана на каждой границе расчетной ячейки и на каждом временном шаге. Процесс нахождения решения задачи Римана основан на спектральном разложении матриц Якоби и является крайне ресурсоемким.

Альтернативным подходом, выбранным в данной дипломной работе, является использование семейства центральных схем высокого разрешения. Фундаментальным преимуществом этих методов является то, что они не требуют решения локальной задачи Римана на границах ячеек. Центральные схемы интегрируют уравнения по вееру волн Римана, опираясь лишь на информацию о максимальной локальной скорости распространения возмущений. Это делает алгоритм полностью алгебраическим, легко векторизуемым и хорошо приспособленным для аппаратного ускорения с использованием выбранной библиотеки.

В рамках данной работы реализованы, протестированы и оптимизированы две центральные схемы из библиотеки *centpy* [9]:

Схема FD2 (Fully-Discrete, 2-го порядка) — полностью дискретная схема Нессяху–Тадмора (Nessyahu–Tadmor) [7]. Является прямым развитием классической схемы Лакса–Фридрихса. Схема использует шахматную сетку для обхода поверхностей разрывов, что позволяет вычислять потоки в гладких областях течения.

Схема SD2 (Semi-Discrete, 2-го порядка) — полудискретная схема Курганова–Тадмора (Kurganov–Tadmor) [8]. Была получена путем перехода

к пределу при $\Delta t \rightarrow 0$ в схеме Нессяху–Тадмора. Использование полудискретной формы позволяет отказаться от шахматных сеток и применять для интегрирования по времени стандартные методы Рунге–Кутты с сохранением TVD-свойств (Total Variation Diminishing).

Вычисление численных потоков в современных многомерных газодинамических комплексах (включая 2D-постановки) базируется на методе расщепления по пространственным координатам. Двумерный оператор дивергенции потока представляется в виде суммы одномерных операторов вдоль осей x и y . В силу этого свойства математический аппарат схем ниже излагается для одномерной скалярной постановки, которая без потери общности обобщается на многомерный векторный случай.

2.2. Полностью дискретный подход

2.2.1. Интегро-интерполяционный метод конечных объемов

Полностью дискретные схемы подразумевают интегрирование законов сохранения одновременно по пространству и по времени. Главная идея центральной схемы Нессяху–Тадмора заключается в интегрировании уравнений Эйлера на так называемой «шахматной» или смещенной сетке. Это позволяет изолировать поверхности разрывов (веера Римана), зарождающиеся на границах ячеек x_j , внутри новых контрольных объемов, где решение остается гладким.

Рассмотрим контрольный объем «шахматной» сетки $[x_j, x_{j+1}]$, смещенный на полшага $\Delta x/2$ относительно исходных узлов. Проинтегрируем уравнение Эйлера (11) по этому объему и по временному интервалу от t^n до t^{n+1} :

$$\int_{x_j}^{x_{j+1}} \int_{t^n}^{t^{n+1}} \left(\frac{\partial U}{\partial t} + \frac{\partial F(U)}{\partial x} \right) dt dx = 0. \quad (44)$$

Применяя формулу Ньютона–Лейбница, вычисляем интегралы:

$$\begin{aligned} \int_{x_j}^{x_{j+1}} U(x, t^{n+1}) dx &= \int_{x_j}^{x_{j+1}} U(x, t^n) dx - \\ &- \int_{t^n}^{t^{n+1}} [F(U(x_{j+1}, t)) - F(U(x_j, t))] dt. \end{aligned} \quad (45)$$

Разделим уравнение (45) на шаг сетки Δx . Левая часть представляет собой среднее значение вектора состояния на новом временном шаге в смещенной ячейке $U_{j+1/2}^{n+1}$. Интеграл от начального состояния (первое слагаемое справа) вычисляется точно за счет кусочно-линейной реконструкции решения $U(x, t^n)$ с наклонами U' :

$$\frac{1}{\Delta x} \int_{x_j}^{x_{j+1}} U(x, t^n) dx = \frac{1}{2}(U_j^n + U_{j+1}^n) + \frac{1}{8}(U_j' - U_{j+1}'). \quad (46)$$

Интеграл потоков по времени (второе слагаемое справа) аппроксимируется правилом средней точки

$$\frac{1}{\Delta x} \int_{t^n}^{t^{n+1}} F(U(x_j, t)) dt \approx \frac{\Delta t}{\Delta x} F(U_j^{n+1/2}), \quad (47)$$

где $U_j^{n+1/2}$ — промежуточное значение (предиктор) в центре исходной ячейки на полушаге по времени.

Подставляя (46) и (47) в уравнение (45), получаем итоговую формулу обновления полностью дискретной центральной схемы FD2:

$$\begin{aligned} U_{j+1/2}^{n+1} &= \frac{1}{2}(U_j^n + U_{j+1}^n) + \frac{1}{8}(U_j' - U_{j+1}') - \\ &- \frac{\Delta t}{\Delta x} \left[F(U_{j+1}^{n+1/2}) - F(U_j^{n+1/2}) \right]. \end{aligned} \quad (48)$$

Формула (48) выражает решение на новом временном слое через две вспомогательные величины, которые на данном этапе считаются известными: наклоны U'_j кусочно-линейной реконструкции и предикторные значения $U_j^{n+1/2}$ на полушаге по времени. Конкретные процедуры их вычисления (реконструкция с ограничителем наклонов и предиктор по схеме Тейлора) детализируются в следующем пункте.

В двумерном пространстве интегрирование выполняется по смещенному контрольному объему $\Omega_{j+1/2,k+1/2} = [x_j, x_{j+1}] \times [y_k, y_{k+1}]$ и временному интервалу $[t^n, t^{n+1}]$. Интеграл от дивергенции потоков $\nabla \cdot (F, G)^T$ по площади ячейки с помощью теоремы Гаусса–Остроградского (формулы Грина) преобразуется в контурный интеграл потока вектора через границу $\partial\Omega$:

$$\iint_{\Omega} \left(\frac{\partial F}{\partial x} + \frac{\partial G}{\partial y} \right) dx dy = \oint_{\partial\Omega} (F dy - G dx). \quad (49)$$

Интегрируя систему уравнений по времени и применяя соотношение (49), получаем полностью дискретный закон сохранения для среднего значения в шахматной ячейке $U_{j+1/2,k+1/2}^{n+1}$. Интеграл от начального состояния (пространственное усреднение по смещенному объему) в двумерном случае вычисляется как среднее по четырем вершинам исходной ячейки с добавлением поправок от градиентов

$$\begin{aligned} \overline{U}_{j+1/2,k+1/2}^n &= \frac{1}{4} (U_{j,k}^n + U_{j+1,k}^n + U_{j,k+1}^n + U_{j+1,k+1}^n) + \\ &+ \frac{1}{16} (U_x^{(L)} - U_x^{(R)}) + \frac{1}{16} (U_y^{(B)} - U_y^{(T)}), \end{aligned} \quad (50)$$

где $U'_{x,y}$ — численные производные вдоль соответствующих направлений на гранях.

Аппроксимируя интегралы потоков во времени по правилу средней точ-

ки, итоговая двумерная схема Нессяху–Тадмора принимает вид:

$$\begin{aligned}
U_{j+1/2,k+1/2}^{n+1} &= \bar{U}_{j+1/2,k+1/2}^n \\
&- \frac{\Delta t}{\Delta x} \left[F \left(U_{j+1,k+1/2}^{n+1/2} \right) - F \left(U_{j,k+1/2}^{n+1/2} \right) \right] \\
&- \frac{\Delta t}{\Delta y} \left[G \left(U_{j+1/2,k+1}^{n+1/2} \right) - G \left(U_{j+1/2,k}^{n+1/2} \right) \right].
\end{aligned} \tag{51}$$

Здесь потоки F и G вычисляются от промежуточных значений предикторов $U^{n+1/2}$, расположенных в центрах граней «шахматной» ячейки.

2.2.2. Алгоритм схемы Нессяху–Тадмора второго порядка

Опираясь на проведённый выше вывод, сформулируем вычислительный алгоритм схемы FD2 второго порядка точности; исторически это первый успешный центральный метод второго порядка, не требующий решения задачи Римана. Ниже детализируются процедуры вычисления наклонов U'_j и предиктора $U_j^{n+1/2}$, которые при выводе формулы обновления (48) считались известными величинами.

Рассмотрим построение схемы для одномерной системы гиперболических уравнений. Введем равномерную вычислительную сетку с шагом по пространству Δx и узлами $x_j = j\Delta x$, а также шаг по времени Δt и моменты времени $t^n = n\Delta t$. Обозначим через U_j^n среднее по ячейке значение вектора консервативных переменных в момент времени t^n .

Алгоритм схемы FD2 состоит из трех основных этапов:

1. Пространственная реконструкция

Для достижения второго порядка точности по пространству конструируется кусочно-линейная аппроксимация решения внутри каждой ячейки:

$$U(x, t^n) = U_j^n + (x - x_j) \frac{U'_j}{\Delta x}, \quad x \in [x_{j-1/2}, x_{j+1/2}], \tag{52}$$

где $U'_j/\Delta x$ — численная производная (наклон) в ячейке j .

При численном решении уравнений гиперболического типа использование линейной реконструкции может приводить к появлению нефизичных осцилляций Годуновского типа вблизи сильных градиентов. Чтобы избежать этого, численная схема должна удовлетворять условию невозрастания полной вариации.

Численная схема обладает TVD-свойством (Total Variation Diminishing), если полная вариация ее решения не возрастает со временем:

$$\mathrm{TV}(U^{n+1}) \leq \mathrm{TV}(U^n), \quad (53)$$

где дискретная полная вариация сеточной функции определяется как сумма модулей разностей значений в соседних ячейках:

$$\mathrm{TV}(U^n) = \sum_j |U_{j+1}^n - U_j^n|. \quad (54)$$

Выполнение данного условия гарантирует, что схема не генерирует новых локальных экстремумов и предотвращает появление паразитных осцилляций на скачках уплотнения.

Для обеспечения TVD-свойства используются нелинейные ограничители наклонов (лимитеры), которые адаптивно уменьшают наклон функции до нуля в точках экстремумов. В данной работе применяется классическая функция `minmod`, которая выбирает наименьшую по модулю разностную производную при совпадении знаков и возвращает нуль в противном случае

$$U'_j = \mathrm{minmod}(U_j^n - U_{j-1}^n, U_{j+1}^n - U_j^n). \quad (55)$$

Математическое определение ограничителя `minmod` для двух аргументов имеет следующий аналитический вид:

$$\mathrm{minmod}(a, b) = \frac{1}{2}(\mathrm{sgn}(a) + \mathrm{sgn}(b)) \cdot \min(|a|, |b|), \quad (56)$$

где $\text{sgn}(x)$ — стандартная знаковая функция:

$$\text{sgn}(x) = \begin{cases} 1, & x > 0, \\ 0, & x = 0, \\ -1, & x < 0. \end{cases} \quad (57)$$

2. Предиктор

Вычисляются промежуточные значения переменных на полшаге по времени $t^{n+1/2} = t^n + \Delta t/2$ в центрах ячеек с использованием разложения в ряд Тейлора

$$U_j^{n+1/2} = U_j^n - \frac{\lambda}{2} F_j', \quad (58)$$

где $\lambda = \frac{\Delta t}{\Delta x}$, а F_j' — численная производная (наклон) потока в ячейке j . В соответствии с исходной формулировкой схемы Нессяху–Тадмора [7] наклон потока вычисляется непосредственно по его разностям — тем же ограничителем minmod , что и наклон решения (55):

$$F_j' = \text{minmod}(F_j^n - F_{j-1}^n, F_{j+1}^n - F_j^n), \quad F_j^n = F(U_j^n). \quad (59)$$

Такой способ вычисления наклона потока не требует построения матрицы Якоби.

3. Корректор

Вычисленные наклоны U_j' и предикторные значения $U_j^{n+1/2}$ подставляются в выведенную ранее формулу обновления (48), дающую решение на следующем временном слое t^{n+1} на смещённой сетке в узлах $x_{j+1/2}$.

Перенос алгоритма Нессяху–Тадмора на двумерные сетки $D(x, y)$ осуществляется путем использования двумерных «шахматных» контрольных объёмов. Вектор состояния U смещается на полшага по обеим осям, переходя из узлов (x_j, y_k) в узлы $(x_{j+1/2}, y_{k+1/2})$.

Пространственная реконструкция (52) расширяется до билинейной ап-

проксимации:

$$U(x, y, t^n) = U_{j,k}^n + (x - x_j) \frac{U'_x}{\Delta x} + (y - y_k) \frac{U'_y}{\Delta y}, \quad (60)$$

где численные производные U'_x и U'_y вычисляются с применением ограничителя (лимитера) независимо по каждому пространственному направлению.

Предиктор вычисляется в центрах ячеек, причём наклоны потоков F'_x и G'_y находятся непосредственно по разностям потоков ограничителем `minmod` независимо вдоль каждого направления (аналогично одномерному случаю):

$$U_{j,k}^{n+1/2} = U_{j,k}^n - \frac{\Delta t}{2\Delta x} F'_x - \frac{\Delta t}{2\Delta y} G'_y. \quad (61)$$

На этапе корректора интегрирование производится по смещённому контрольному объёму $[x_j, x_{j+1}] \times [y_k, y_{k+1}]$, что приводит к выведенной ранее двумерной формуле обновления (51).

Главным недостатком полностью дискретной схемы FD2 является необходимость работы на «шахматной» сетке. Это вызывает существенные трудности при постановке граничных условий, так как границы сетки смещаются между временными шагами, что делает метод неприменимым для стационарных задач из-за сильной числовой диссипации, зависящей от шага по времени Δt . При двумерной постановке дополнительно возрастает алгоритмическая сложность при вычислении угловых потоков и постановке граничных условий, из-за чего в современных 2D-симуляторах чаще используются полудискретные аналоги. Эти проблемы были успешно решены в полудискретных схемах, рассматриваемых далее.

2.3. Полудискретный подход

2.3.1. Полудискретная форма уравнений

Основой для построения схем семейства SD2 и SD3 является метод конечных объемов в его полудискретной формулировке, метод линий. В отличие

от полностью дискретных схем, данный подход подразумевает интегрирование уравнений только по пространственным координатам, оставляя время непрерывной переменной.

В качестве расчетной области выберем одномерную равномерную сетку с шагом Δx :

$$\Omega_h = \{x_j : x_j = j\Delta x\}, \quad j = 0, 1, \dots, N. \quad (62)$$

Определим границы контрольного объема как точки

$$x_{j-1/2} = x_j - \frac{\Delta x}{2} \quad \text{и} \quad x_{j+1/2} = x_j + \frac{\Delta x}{2}. \quad (63)$$

Проинтегрируем исходное уравнение Эйлера в дивергентной форме (11) по контрольному объему $[x_{j-1/2}, x_{j+1/2}]$:

$$\int_{x_{j-1/2}}^{x_{j+1/2}} \frac{\partial U(x, t)}{\partial t} dx + \int_{x_{j-1/2}}^{x_{j+1/2}} \frac{\partial F(U(x, t))}{\partial x} dx = 0. \quad (64)$$

Поскольку границы ячейки не зависят от времени, производную по времени в первом слагаемом можно вынести за знак интеграла. Ко второму слагаемому применим формулу Ньютона–Лейбница

$$\frac{d}{dt} \left(\int_{x_{j-1/2}}^{x_{j+1/2}} U(x, t) dx \right) + [F(U(x_{j+1/2}, t)) - F(U(x_{j-1/2}, t))] = 0. \quad (65)$$

Введем понятие пространственного среднего значения вектора консервативных переменных в j -й ячейке

$$v_j(t) = \frac{1}{\Delta x} \int_{x_{j-1/2}}^{x_{j+1/2}} U(x, t) dx. \quad (66)$$

Разделив уравнение (65) на шаг сетки Δx и подставив определение (66),

получаем строгую полудискретную форму закона сохранения

$$\frac{dv_j(t)}{dt} = -\frac{H_{j+1/2}(t) - H_{j-1/2}(t)}{\Delta x}. \quad (67)$$

Уравнение (67) представляет собой систему обыкновенных дифференциальных уравнений относительно средних значений $v_j(t)$. Здесь $H_{j+1/2}(t)$ обозначает численный поток через границу ячейки, который аппроксимирует физический поток $F(U(x_{j+1/2}, t))$.

В отличие от полностью дискретных методов, которые требуют совместного интегрирования по пространству и времени, полудискретная форма (67) позволяет разделить задачу на два независимых этапа: вычисление численных потоков $H_{j\pm 1/2}(t)$ на основе пространственной реконструкции и интегрирование полученной системы ОДУ по времени с использованием высокоточных TVD-методов Рунге–Кутты.

В случае двух пространственных измерений контрольным объемом выступает прямоугольная ячейка $\Omega_{j,k} = [x_{j-1/2}, x_{j+1/2}] \times [y_{k-1/2}, y_{k+1/2}]$. Интегрируя двумерное уравнение Эйлера (5) по этому объему и применяя формулу Грина к дивергентному члену $\left(\frac{\partial F}{\partial x} + \frac{\partial G}{\partial y}\right)$, мы получаем:

$$\begin{aligned} & \frac{d}{dt} \iint_{\Omega_{j,k}} U(x, y, t) dx dy = \\ & = - \int_{y_{k-1/2}}^{y_{k+1/2}} \left[F|_{x_{j+1/2}} - F|_{x_{j-1/2}} \right] dy - \\ & \quad - \int_{x_{j-1/2}}^{x_{j+1/2}} \left[G|_{y_{k+1/2}} - G|_{y_{k-1/2}} \right] dx. \end{aligned} \quad (68)$$

Вводя пространственное среднее значение

$$v_{j,k}(t) = \frac{1}{\Delta x \Delta y} \iint U dx dy \quad (69)$$

и деля уравнение на площадь ячейки, мы получаем двумерную систему ОДУ:

$$\frac{dv_{j,k}(t)}{dt} = -\frac{H_{j+1/2,k}^x - H_{j-1/2,k}^x}{\Delta x} - \frac{H_{j,k+1/2}^y - H_{j,k-1/2}^y}{\Delta y}, \quad (70)$$

где H^x и H^y — численные аппроксимации линейных интегралов физических потоков вдоль соответствующих граней ячейки.

2.3.2. Алгоритм схемы Курганова–Тадмора второго порядка с линейной реконструкцией

Несмотря на высокую эффективность схемы Нессяху–Тадмора, использование ею шахматных сеток создает значительные алгоритмические трудности при задании сложных граничных условий и делает диссипацию схемы зависимой от шага по времени Δt . Для решения этих проблем А. Курганов и Э. Тадмор предложили полудискретный предел центральной схемы [8]. Если в схеме FD2 устремить шаг по времени к нулю ($\Delta t \rightarrow 0$), пространственная ширина «смещённых» ячеек Римана стягивается в линию, что позволяет полностью отказаться от использования шахматных сеток и вычислять потоки на фиксированной (исходной) сетке.

В результате такого предельного перехода вновь приходим к полудискретной форме закона сохранения (67), выведенной выше прямым интегрированием по контрольному объёму, — системе обыкновенных дифференциальных уравнений по времени относительно средних по ячейкам значений $v_j(t)$. Специфику схемы SD2 определяет конкретный вид численного потока Курганова–Тадмора $H_{j+1/2}(t)$ через грань ячейки $x_{j+1/2}$, который при выводе (67) считался известной величиной.

Ключевым элементом схемы SD2 является конструкция этого потока. Он вычисляется как центральная аппроксимация физических потоков с добавлением диссипативного члена (числовой вязкости), который пропорцио-

нален максимальной локальной скорости распространения волн $a_{j+1/2}$:

$$H_{j+1/2} = \frac{F(v_{j+1/2}^+) + F(v_{j+1/2}^-)}{2} - \frac{a_{j+1/2}}{2} [v_{j+1/2}^+ - v_{j+1/2}^-]. \quad (71)$$

Здесь $v_{j+1/2}^+$ и $v_{j+1/2}^-$ — реконструированные значения консервативных переменных на правой и левой стороне грани $x_{j+1/2}$ соответственно. Реконструкция выполняется с использованием ограничителя наклонов (например, minmod) для сохранения свойства невозрастания полной вариации (TVD):

$$v_{j+1/2}^- = v_j(t) + \frac{\Delta x}{2}(v_x)_j(t), \quad v_{j+1/2}^+ = v_{j+1}(t) - \frac{\Delta x}{2}(v_x)_{j+1}(t), \quad (72)$$

где $(v_x)_j$ — численная производная (наклон) в j -й ячейке.

Скалярная локальная скорость распространения возмущений $a_{j+1/2}(t)$ оценивается через спектральный радиус ρ матрицы Якоби $A(v) = \frac{\partial F}{\partial v}$ по формуле:

$$a_{j+1/2}(t) = \max \left[\rho \left(A(v_{j+1/2}^-) \right), \rho \left(A(v_{j+1/2}^+) \right) \right]. \quad (73)$$

Для системы уравнений Эйлера спектральный радиус матрицы Якоби равен сумме локальной скорости потока и местной скорости звука: $\rho(A) = |u| + c$.

Поскольку система (67) представляет собой систему ОДУ, интегрирование по времени осуществляется методами Рунге–Кутты. Для сохранения свойства TVD необходимо использовать специальные SSP-методы (Strong Stability Preserving). В данной работе применяется SSP-метод Рунге–Кутты 2-го порядка [14], согласованный по порядку точности с пространственной аппроксимацией схемы SD2:

$$\begin{aligned} v^{(1)} &= v^n + \Delta t L(v^n), \\ v^{n+1} &= \frac{1}{2}v^n + \frac{1}{2}v^{(1)} + \frac{1}{2}\Delta t L(v^{(1)}), \end{aligned} \quad (74)$$

где $L(v)$ — правая часть уравнения (67), а Δt выбирается из условия Куранта–

Фридрихса–Леви (CFL).

Таким образом, полудискретная схема SD2 полностью избавляется от шахматных сеток, сохраняя при этом «Riemann-solver-free» природу: для вычисления потока требуется знать лишь максимальную скорость волны $a_{j+1/2}$, без необходимости точного разрешения структуры веера Римана.

Одно из главных достоинств полудискретного подхода Курганова–Тадмора — математически строгий и алгоритмически прозрачный переход к многомерным задачам. Двумерная полудискретная система (70), выведенная выше, сохраняет ту же структуру.

Потоки H^x и H^y вычисляются независимо (dimensional splitting). Поток $H_{j+1/2,k}^x$ вдоль оси x рассчитывается по формуле (71) с использованием значений $v_{j+1/2,k}^+$ и $v_{j+1/2,k}^-$, полученных путем одномерной реконструкции вдоль индексов j . Аналогично, поток $H_{j,k+1/2}^y$ вдоль оси y вычисляется с использованием реконструкции вдоль индексов k :

$$H_{j,k+1/2}^y = \frac{G(v_{j,k+1/2}^+) + G(v_{j,k+1/2}^-)}{2} - \frac{b_{j,k+1/2}}{2} [v_{j,k+1/2}^+ - v_{j,k+1/2}^-], \quad (75)$$

где

$$b_{j,k+1/2} = \max(\rho(B(v^-)), \rho(B(v^+))) \quad (76)$$

— максимальная скорость распространения волн вдоль оси y (определяемая через спектральный радиус матрицы Якоби $B(v) = \partial G / \partial v$).

Благодаря аддитивной структуре правой части уравнения (70), интегратор по времени Рунге–Кутты (74) применяется к двумерному массиву состояний без каких-либо алгоритмических изменений.

3. Программная реализация

3.1. Библиотека для вычислений JAX

Библиотека JAX [5] представляет собой современный программный комплекс для высокопроизводительных численных вычислений, разработанный компанией Google. В его основе лежит компиляция Python-кода в оптимизированные инструкции для гетерогенных вычислительных устройств (CPU, GPU, TPU) с использованием компилятора XLA (Accelerated Linear Algebra). Ключевой особенностью JAX является опора на парадигму функционального программирования, требующую применения чистых функций без побочных эффектов.

В рамках данной работы из инструментария JAX непосредственно использованы следующие возможности:

- JIT-компиляция (Just-In-Time). Декоратор `jax.jit` выполняет трассировку Python-функции и компилирует ее в оптимизированный код XLA, устраняя накладные расходы интерпретатора Python и обеспечивая слияние операций (operator fusion). В разработанном комплексе под `jax.jit` помещается полный шаг интегрирования по времени.
- Векторизованные операции над массивами. Модуль `jax.numpy` предоставляет интерфейс в стиле NumPy, позволяющий выражать вычисления на структурированных сетках через матричные срезы и сдвиги осей (`jax.numpy.moveaxis`) без явных циклов по пространству.
- Функциональное обновление массивов. Поскольку массивы JAX неизменяемы, запись в элементы выполняется через конструкцию `at[...].set(...)`, возвращающую новый массив; этот механизм применяется для сохранения временных слоёв решения.
- Структурный цикл `jax.lax.while_loop`. Позволяет включить итерационный процесс продвижения по времени в компилируемый граф вычислений, что недостижимо при использовании обычного цикла Python.

- Перенос вычислений между устройствами. Один и тот же исходный код выполняется на CPU и GPU без изменений, что использовано при анализе аппаратного ускорения.

Перечисленные возможности определяют структуру и стиль реализации программного комплекса, рассматриваемого далее.

3.2. Функциональная парадигма JAX

Эффективное применение JIT-компиляции и переноса вычислений на ускорители требует отказа от объектно-ориентированного стиля с изменяемым состоянием в пользу функционального. В исходной библиотеке *centpy* [9] использовались объекты-решатели, хранящие и изменяющие внутреннее состояние в процессе счёта. Такой подход несовместим с требованиями *jax.jit*, для которого компилируемая функция должна быть чистой, то есть детерминированно отображать аргументы в результат без модификации внешних данных.

В связи с этим расчётное ядро комплекса *jax_centpy* построено в функциональном стиле, опирающемся на три принципа:

1. Разделение данных и поведения. Параметры сетки и описание физической задачи (функции потоков, спектральных радиусов, граничных условий) вынесены в неизменяемые структуры *NamedTuple* (*Pars1d*, *Pars2d*, *Equation1d*, *Equation2d*) и передаются в алгоритмы как аргументы.
2. Отсутствие побочных эффектов. Функции численных схем и шага по времени не изменяют переданные им массивы: они принимают текущее состояние *u* и возвращают новый массив, что делает их пригодными для компиляции *jax.jit*.
3. Векторизация вместо циклов. Вычисление потоков на гранях ячеек выражено через срезы массивов и сдвиги осей, благодаря чему схемы

SD2 и FD2 записываются как последовательность векторных операций.

Подчинение этим принципам делает расчётное ядро прозрачным для компилятора `jah.jit` и позволяет исполнять его на графическом ускорителе без изменения исходного кода.

3.3. Архитектура программного комплекса

Комплекс `jah_centru` имеет модульную структуру, в которой каждый модуль отвечает за отдельный уровень абстракции. Такое разделение обеспечивает расширяемость: добавление новых уравнений или ограничителей не требует изменения кода решателя. Взаимодействие модулей организовано следующим образом:

- `core.py` — базовый уровень данных. Содержит определения неизменяемых структур `Pars1d`, `Pars2d`, `Equation1d`, `Equation2d`, задающих параметры сетки и интерфейсы функций физических потоков, спектральных радиусов и обработки границ. Здесь же включается режим 64-битной арифметики с плавающей точкой, необходимый для задач газовой динамики.
- `limiters.py` — уровень базовых операций. Содержит набор ограничителей наклона (`minmod`, `superbee`, `van_leer` и другие), подавляющих нефизичные осцилляции вблизи разрывов.
- `schemes.py` и `time_integration.py` — уровень численных методов. В `schemes.py` реализованы кусочно-линейная реконструкция и вычисление правых частей (численных потоков) полудискретной схемы SD2, а также полный шаг полностью дискретной схемы FD2 для одномерной и двумерной постановок. Модуль `time_integration.py` содержит чистые функции интеграторов по времени; в расчётах используется метод SSP-RK2, согласованный по порядку точности с пространственной аппроксимацией.

- `solver.py` — уровень управления вычислением. Содержит используемые в работе решатели `Solver1d` и `Solver2d`. Каждый из них настраивается на схему (SD2 или FD2) и ограничитель, инициализирует сетку, помещает шаг интегрирования под `jax.jit` и организует продвижение по времени через `jax.lax.while_loop` с адаптивным выбором шага Δt по числу Куранта. В ходе счёта решатель сохраняет все промежуточные временные слои, что используется для последующего анализа и визуализации.
- `equations.py` — уровень физических моделей. Содержит фабричные функции, формирующие конкретные постановки задач: одномерное и двумерное уравнения Эйлера, включая модифицированную задачу о распаде разрыва с областью локального энерговложения.
- `boundaries.py` — уровень граничных условий. Содержит набор обработчиков границ расчётной области, реализуемых через добавление фиктивных ячеек: периодические условия, условия свободного выхода (Неймана), условия Дирихле и условие непротекания на твёрдой стенке для одномерной и двумерной постановок.

Представленная архитектура обеспечивает перенос расчётов на графический ускоритель при сохранении читаемости математического кода. Передача конфигурации через структуры `Equation` делает комплекс применимым к широкому классу гиперболических систем.

4. Численное исследование

4.1. Верификация численных методов

Для подтверждения корректности разработанной JAX-реализации схем $SD2$ и $FD2$ проводится численный анализ сходимости на гладких тестовых решениях. В отличие от задач с разрывами, рассматриваемых в разделе 4.2, где порядок схемы деградирует до первого в окрестности ударных волн, гладкие решения позволяют наблюдать теоретический порядок аппроксимации схемы в глобальной норме.

Для количественной оценки точности используются нормы L_1 и L_2 . Норма L_1 чувствительна к погрешностям интегрального характера и является стандартной метрикой в вычислительной газодинамике:

$$\|e\|_{L_1} = \frac{1}{N} \sum_{i=1}^N |u_i^h - u_i^{\text{ref}}|, \quad (77)$$

Норма L_2 или среднеквадратическая позволяет дополнительно оценить равномерность распределения погрешности по всей области:

$$\|e\|_{L_2} = \sqrt{\frac{1}{N} \sum_{i=1}^N (u_i^h - u_i^{\text{ref}})^2}, \quad (78)$$

где в обоих выражениях u_i^h — численное решение в центре i -й ячейки сетки с числом ячеек N , u_i^{ref} — точное аналитическое решение.

Эмпирический порядок сходимости при последовательном удвоении числа ячеек вычисляется по формуле Ричардсона:

$$p = \log_2 \frac{\|e^{(N)}\|}{\|e^{(2N)}\|}, \quad (79)$$

где $\|e^{(N)}\|$ и $\|e^{(2N)}\|$ — ошибки на грубой и вдвое измельченной сетке в выбранной норме соответственно. Согласно теоретическому порядку аппроксимации, схемы $SD2$ и $FD2$ должны демонстрировать $p \approx 2$.

Одномерный случай

В качестве одномерной тестовой задачи рассматривается уравнение Эйлера (11) на периодической области $x \in [0, 1]$ с начальными условиями в виде синусоидального возмущения плотности при постоянных скорости и давлении:

$$\rho(x, 0) = 1 + 0.2 \sin(2\pi x), \quad u(x, 0) = 1, \quad p(x, 0) = 1. \quad (80)$$

Точное решение представляет собой перенос начального профиля без изменения формы со скоростью $u = 1$, что при периодических граничных условиях даёт:

$$\rho(x, t) = 1 + 0.2 \sin(2\pi(x - t)). \quad (81)$$

Расчёт проводился до момента времени $t = 1$ (один полный период переноса) при числе Куранта $C = 0.475$ с ограничителем наклонов типа монотонизированной центральной разности (mc). Ошибка вычислялась по плотности ρ в соответствии с формулами (77)–(78). Для оценки сходимости последовательно использовались сетки с числом ячеек $N \in \{50, 100, 200, 400\}$.

Результаты анализа сходимости для схем *SD2* и *FD2* в одномерном случае представлены в таблицах 1–2.

Таблица 1. Сходимость схемы *SD2* относительно точного решения, одномерный случай, плотность ρ

N	$\ e\ _{L_1}$	p_{L_1}	$\ e\ _{L_2}$	p_{L_2}
50	1.9348×10^{-3}	—	2.2379×10^{-3}	—
100	5.5983×10^{-4}	1.79	6.9353×10^{-4}	1.69
200	1.4718×10^{-4}	1.93	2.1407×10^{-4}	1.70
400	3.7337×10^{-5}	1.98	6.6111×10^{-5}	1.70

Как следует из таблиц 1–2, обе схемы демонстрируют близкий ко второму порядок сходимости: по интегральной норме L_1 эмпирический порядок выходит на $p \approx 2$, тогда как по среднеквадратичной норме L_2 для схемы *SD2* он несколько ниже ($p \approx 1.7$). Умеренное снижение наблюдаемого порядка в более строгих нормах на гладких решениях является характерной

Таблица 2. Сходимость схемы $FD2$ относительно точного решения, одномерный случай, плотность ρ

N	$\ e\ _{L_1}$	p_{L_1}	$\ e\ _{L_2}$	p_{L_2}
50	1.3469×10^{-3}	—	1.8671×10^{-3}	—
100	3.2891×10^{-4}	2.03	5.2057×10^{-4}	1.84
200	7.5657×10^{-5}	2.12	1.4457×10^{-4}	1.85
400	1.7951×10^{-5}	2.08	4.0688×10^{-5}	1.83

особенностью центральных схем рассматриваемого семейства, обусловленной диссипативной структурой потоковой функции Лакса–Фридрихса; оно не связано с выбором ограничителя наклонов и не свидетельствует об ошибке реализации. Формальная второпорядочность реализованных операторов пространственной реконструкции и интегрирования по времени методами TVD–Рунге–Кутты подтверждается двумерным тестом на изоэнтропийном вихре, где порядок $p \approx 2$ достигается по всем примитивным переменным.

Вместе с тем следует отметить, что проверка TVD-свойства на гладком решении уравнений Эйлера носит иллюстративный характер: для систем уравнений полная вариация отдельных компонент не обязана убывать в силу нелинейного взаимодействия волн [12]. Строгое теоретическое обоснование TVD-условия справедливо лишь для скалярных законов сохранения. Поэтому для независимой проверки монотонности схем дополнительно рассматривается одномерное уравнение Бюргерса.

Проверка TVD-свойства на уравнении Бюргерса (1D)

Рассматривается скалярное нелинейное уравнение Бюргерса на периодической области $x \in [0, 2\pi]$:

$$\frac{\partial u}{\partial t} + \frac{\partial}{\partial x} \left(\frac{u^2}{2} \right) = 0, \quad u(x, 0) = u_0(x) = \sin x + \frac{1}{2} \sin \frac{x}{2}. \quad (82)$$

Выбор данной задачи обусловлен двумя её свойствами. Во-первых, при $t < T_*$, где

$$T_* = -\frac{1}{\min_x u'_0(x)} \approx 0.99 \quad (83)$$

— момент опрокидывания характеристик, решение остаётся гладким и описывается

$$u(x, t) = u_0(\xi), \quad \xi + u_0(\xi) t = x, \quad u_0(\xi) = \sin \xi + \frac{1}{2} \sin \frac{\xi}{2}, \quad (84)$$

что позволяет находить ξ итерационно методом Ньютона. Во-вторых, при $t > T_*$ в решении формируется ударная волна, и полная вариация точного решения начинает монотонно убывать [12]. Именно в этом разрывном режиме для скалярного уравнения сохранения строго доказано выполнение TVD-условия (53) [13].

Расчёт проводился до момента $t = 2$, что существенно превышает время опрокидывания T_* и позволяет проследить как гладкую стадию переноса, так и формирование с последующим распространением ударной волны, при числе Куранта $C = 0.475$.

На рисунке 3 показана эволюция полной вариации $TV(t)$ для обеих схем.

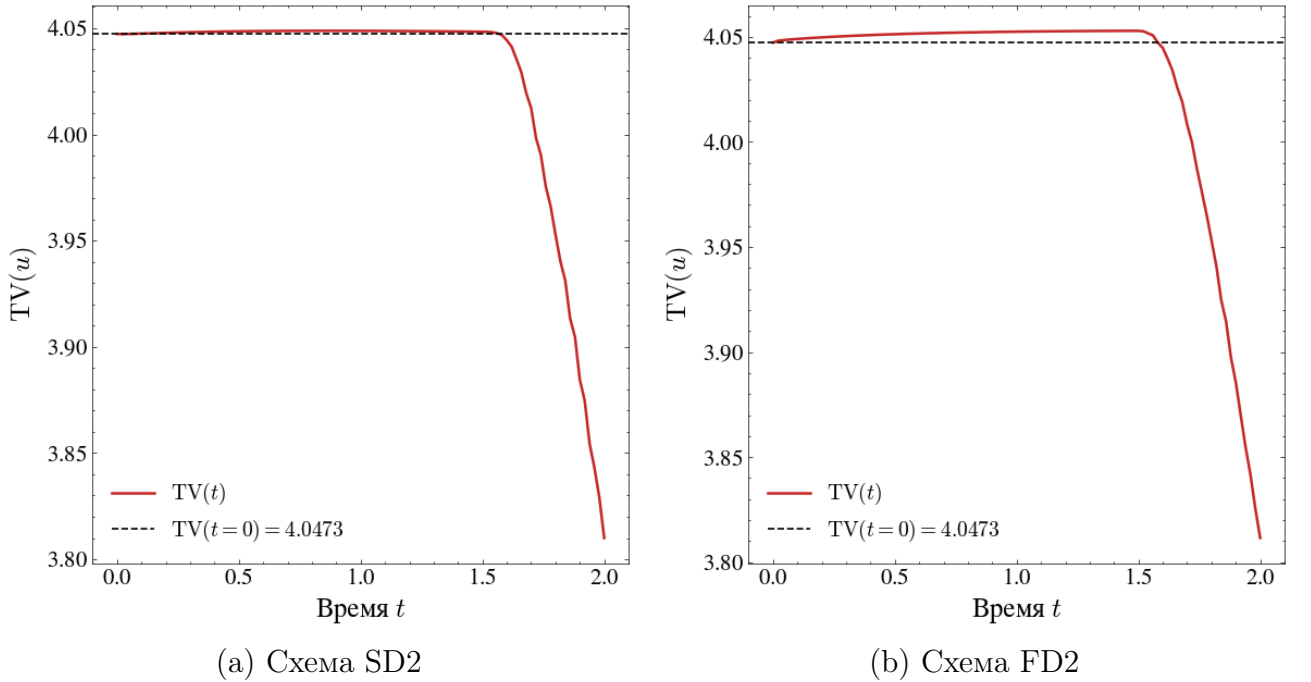


Рисунок 3. Эволюция полной вариации $TV(t)$ для уравнения Бюргерса (82). Пунктирная линия соответствует начальному значению $TV(t = 0)$.

Как следует из рисунка 3, на начальном этапе расчёта полная вари-

ация остаётся практически постоянной: гладкий профиль переносится без заметной диссипации. После образования и развития ударной волны $TV(t)$ начинает монотонно убывать. Для схемы $SD2$ полная вариация не превышает начального значения на всём протяжении расчёта; для схемы $FD2$ на гладкой стадии наблюдается пренебрежимо малое (менее 0.2 %) превышение уровня $TV(t = 0)$, не сопровождающееся ростом вариации в окрестности разрыва. Это подтверждает, что применяемый лимитер `minmod` совместно с TVD-интегратором Рунге–Кутты [14] корректно подавляет осцилляции вблизи разрыва и обеспечивает монотонность численного решения.

Двумерный случай

Для верификации двумерной реализации используется классический тест — изоэнтропийный вихрь Ху и Шу [15], обладающий точным аналитическим решением и широко применяемый в вычислительной газодинамике для оценки порядка схем на многомерных сетках. Задача формулируется на квадратной области $[0, 10] \times [0, 10]$ с периодическими граничными условиями по обоим направлениям.

Начальные условия задаются как гладкое возмущение фонового однородного потока $(\rho_\infty, u_\infty, v_\infty, p_\infty) = (1, 1, 1, 1)$ вихревым возмущением интенсивности $\varepsilon = 5$ с центром в точке $(x_0, y_0) = (5, 5)$:

$$\begin{aligned}\delta u &= -\frac{\varepsilon}{2\pi}(y - y_0) e^{\frac{1-r^2}{2}}, \\ \delta v &= +\frac{\varepsilon}{2\pi}(x - x_0) e^{\frac{1-r^2}{2}}, \\ \delta T &= -\frac{(\gamma - 1)\varepsilon^2}{8\gamma\pi^2} e^{1-r^2},\end{aligned}\tag{85}$$

где $r^2 = (x - x_0)^2 + (y - y_0)^2$. Плотность и давление восстанавливаются из условия изоэнтропийности течения:

$$\rho = (1 + \delta T)^{\frac{1}{\gamma-1}}, \quad p = (1 + \delta T)^{\frac{\gamma}{\gamma-1}}.\tag{86}$$

Точное решение — перенос вихря без изменения его структуры с единичной скоростью по обеим осям (в силу периодичности при $t = 10$ вихрь возвращается в исходное положение):

$$(\rho, u, v, p)(x, y, t) = (\rho, u, v, p)(x - t, y - t, 0). \quad (87)$$

Расчёт проводился при числе Куранта $C = 0.475$ на квадратных равномерных сетках $N \times N$, $N \in \{50, 100, 200, 400\}$, с центральной (неограниченной) реконструкцией наклонов. Погрешность вычислялась относительно сдвинутого аналитического решения (87) на конечном временном слое $t = 1$.

Результаты анализа сходимости представлены в таблицах 3–4.

Полученные результаты свидетельствуют о том, что двумерная JAX-реализация корректно воспроизводит теоретический второй порядок аппроксимации схем по всем примитивным переменным. Таким образом, операторы расщепления по направлениям, реализованные с применением тензорных операций `jax.numpy`, сохраняют расчетную точность многомерной схемы.

Перенос верификации монотонности на двумерный случай требует принципиальной оговорки. Теорема Гудмана–Левека [16] устанавливает, что для многомерных задач не существует линейных схем второго и выше порядка точности, которые были бы TVD в классическом смысле (53).

Попытка применить условие

$$\text{TV}(u^{n+1}) \leq \text{TV}(u^n) \quad (88)$$

напрямую к двумерной задаче приводит к принципиальным нарушениям даже для монотонных схем, поскольку перенос разрыва под углом к сетке неизбежно порождает осциллирующий вклад в суммарную вариацию $\text{TV} = \text{TV}_x + \text{TV}_y$.

Таблица 3. Анализ сходимости схемы $SD2$, двумерный случай

$N \times N$	ρ				u			
	$\ e^{(N)}\ _{L_1}$	p_{L_1}	$\ e^{(N)}\ _{L_2}$	p_{L_2}	$\ e^{(N)}\ _{L_1}$	p_{L_1}	$\ e^{(N)}\ _{L_2}$	p_{L_2}
50×50	6.483×10^{-4}	—	1.466×10^{-3}	—	1.399×10^{-3}	—	3.418×10^{-3}	—
100×100	1.294×10^{-4}	2.32	2.907×10^{-4}	2.33	2.774×10^{-4}	2.33	6.879×10^{-4}	2.31
200×200	2.817×10^{-5}	2.20	6.354×10^{-5}	2.19	6.100×10^{-5}	2.18	1.483×10^{-4}	2.21
400×400	6.700×10^{-6}	2.07	1.522×10^{-5}	2.06	1.466×10^{-5}	2.06	3.506×10^{-5}	2.08
$N \times N$	v				p			
	$\ e^{(N)}\ _{L_1}$	p_{L_1}	$\ e^{(N)}\ _{L_2}$	p_{L_2}	$\ e^{(N)}\ _{L_1}$	p_{L_1}	$\ e^{(N)}\ _{L_2}$	p_{L_2}
50×50	1.416×10^{-3}	—	3.393×10^{-3}	—	7.701×10^{-4}	—	1.948×10^{-3}	—
100×100	2.776×10^{-4}	2.35	6.681×10^{-4}	2.34	1.578×10^{-4}	2.29	3.788×10^{-4}	2.36
200×200	5.994×10^{-5}	2.21	1.412×10^{-4}	2.24	3.574×10^{-5}	2.14	8.254×10^{-5}	2.20
400×400	1.417×10^{-5}	2.08	3.306×10^{-5}	2.09	8.626×10^{-6}	2.05	1.967×10^{-5}	2.07

Таблица 4. Анализ сходимости схемы $FD2$, двумерный случай

$N \times N$	ρ				u			
	$\ e^{(N)}\ _{L_1}$	p_{L_1}	$\ e^{(N)}\ _{L_2}$	p_{L_2}	$\ e^{(N)}\ _{L_1}$	p_{L_1}	$\ e^{(N)}\ _{L_2}$	p_{L_2}
50×50	1.500×10^{-3}	—	3.389×10^{-3}	—	3.933×10^{-3}	—	9.460×10^{-3}	—
100×100	1.740×10^{-4}	3.11	4.144×10^{-4}	3.03	4.058×10^{-4}	3.28	1.050×10^{-3}	3.17
200×200	3.089×10^{-5}	2.49	7.330×10^{-5}	2.50	7.565×10^{-5}	2.42	1.899×10^{-4}	2.47
400×400	6.365×10^{-6}	2.28	1.503×10^{-5}	2.29	1.632×10^{-5}	2.21	3.959×10^{-5}	2.26
$N \times N$	v				p			
	$\ e^{(N)}\ _{L_1}$	p_{L_1}	$\ e^{(N)}\ _{L_2}$	p_{L_2}	$\ e^{(N)}\ _{L_1}$	p_{L_1}	$\ e^{(N)}\ _{L_2}$	p_{L_2}
50×50	3.929×10^{-3}	—	9.370×10^{-3}	—	1.837×10^{-3}	—	5.168×10^{-3}	—
100×100	3.919×10^{-4}	3.33	9.813×10^{-4}	3.26	1.993×10^{-4}	3.20	5.186×10^{-4}	3.32
200×200	6.910×10^{-5}	2.50	1.648×10^{-4}	2.57	3.625×10^{-5}	2.46	9.010×10^{-5}	2.52
400×400	1.371×10^{-5}	2.33	3.171×10^{-5}	2.38	7.763×10^{-6}	2.22	1.824×10^{-5}	2.30

Практически значимой заменой TVD-свойства в многомерном случае служит *принцип максимума*: численное решение не должно порождать новые экстремумы, превышающие начальные [12]:

$$\min_j u_j^0 \leq u_j^n \leq \max_j u_j^0 \quad \forall j, n. \quad (89)$$

Это условие выполняется для монотонных схем в любом числе измерений и является стандартным критерием верификации монотонности в вычислительной газодинамике [12].

Проверка принципа максимума на задаче линейной адвекции (2D)

Рассматривается двумерное уравнение линейной адвекции на периодической области $[0, 1]^2$ с единичными скоростями по обоим направлениям:

$$\frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} + \frac{\partial u}{\partial y} = 0, \quad u(x, y, 0) = \begin{cases} 1, & \sqrt{(x - 0.5)^2 + (y - 0.5)^2} < 0.25, \\ 0, & \text{иначе.} \end{cases} \quad (90)$$

Начальное условие представляет собой «шапку» — окружность единичной высоты с центром в точке $(0.5, 0.5)$ и радиусом 0.25. Граница «шапки» является разрывом первого рода, что позволяет наблюдать работу лимитера в двумерном случае.

Точное решение — перенос «шапки» без изменения формы:

$$u(x, y, t) = u_0((x - t) \bmod 1, (y - t) \bmod 1). \quad (91)$$

Данная задача служит эталонной для верификации принципа максимума: численное решение обязано удовлетворять условию (89) в каждый момент времени. Расчёт проводился до момента $t = 1$ (один полный период переноса) при числе Куранта $C = 0.45$ на сетке $N \times N$, $N = 100$.

На рисунке 4 показана эволюция $u_{\min}(t)$ и $u_{\max}(t)$ на протяжении рас-

чёта для обеих схем.

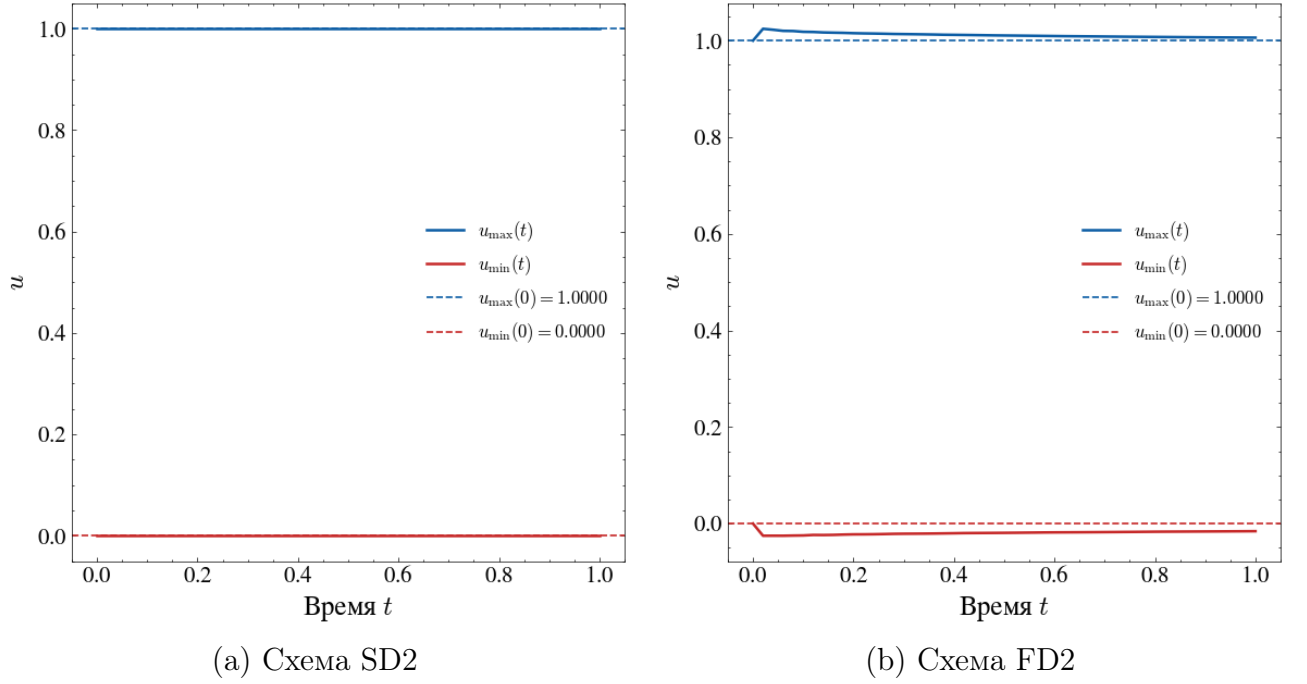


Рисунок 4. Эволюция $u_{\min}(t)$ и $u_{\max}(t)$ для задачи линейной адвекции (90). Пунктирные линии — начальные значения $u_{\min}(t = 0) = 0$ и $u_{\max}(t = 0) = 1$.

Как следует из рисунка 4, схема *SD2* строго удовлетворяет условию (89) на протяжении всего расчёта: $u_{\min}(t) \geq 0$ и $u_{\max}(t) \leq 1$ для всех $t \in [0, 1]$.

Схема *FD2* демонстрирует незначительные нарушения в начальный момент расчёта: undershoot по минимуму ($u_{\min} \approx -0.02$) и сопоставимый по величине overshoot по максимуму ($u_{\max} \approx 1.01$). Они обусловлены особенностью шахматной пространственной сетки Нессияху–Тадмора: при инициализации разрыва чётные и нечётные ячейки сдвинуты относительно друг друга на половину шага, что на первых итерациях приводит к локальному нарушению монотонности. В дальнейшем отклонения не нарастают, а монотонно затухают: overshoot по максимуму исчезает уже к $t \approx 0.1$, а остаточный undershoot по минимуму ($u_{\min} \approx -0.01$) плавно уменьшается по модулю и к концу расчёта ($t = 1$) практически обращается в нуль. Описанное поведение является характерной особенностью полностью дискретных разностных схем на шахматных сетках при наличии разрывов [12], имеет ограниченную по модулю амплитуду и не приводит к накоплению погрешности в ходе расчёта.

При уменьшении числа Куранта до $C = 0.27$ (рисунок 5) нарушение полностью исчезает, что подтверждает зависимость диссипативных свойств $FD2$ от шага по времени.

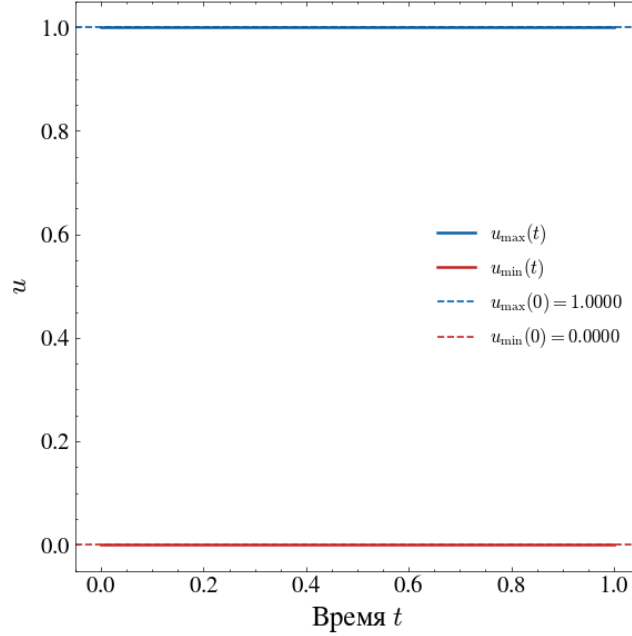


Рисунок 5. Эволюция $u_{\min}(t)$ и $u_{\max}(t)$ для задачи линейной адвекции (90). Схема $FD2$, число Куранта $C = 0.27$

Проведённый анализ сходимости и проверка TVD-свойства для одномерных и двумерных задач подтверждают корректность разработанной JAX-реализации схем $SD2$ и $FD2$. Второй порядок сходимости подтверждён на гладких задачах: в одномерном случае (контактная волна) — по интегральной норме L_1 , а на двумерном изоэнтропийном вихре, возбуждающем все характеристические поля, — $p \approx 2$ по всем примитивным переменным и обеим нормам. Монотонность численного решения верифицирована на скалярных задачах с разрывами: в одномерном случае — через TVD-условие на уравнении Бюргерса (53), в двумерном — через принцип максимума (89) на задаче линейной адвекции, поскольку строгое TVD-свойство в многомерных задачах недостижимо для схем выше первого порядка [16].

4.2. Анализ модифицированных задач

В данном разделе проводится детальный анализ результатов численного моделирования модифицированных задач газовой динамики. Основное внимание уделяется исследованию процессов взаимодействия сильных ударных волн с областями локального понижения плотности, моделирующими тепловой след зоны энерговложения [1], а также сравнению разрешающей способности схем SD2 и FD2 на сложных нестационарных течениях. Рассматривается как одномерная постановка, позволяющая изучить волновую структуру в направлении распространения фронта, так и двумерная задача, демонстрирующая эффекты рефракции на границах раздела сред различной формы (круглом и квадратном пузырях). Вычисления проводились с числом Куранта $CFL = 0.475$ до конечного момента времени $t_{\text{end}} = 0.4$. В одномерном случае использовалась равномерная сетка из $N = 400$ ячеек, в двумерном — сетка 200×200 ячеек.

Физическая картина течения: одномерная задача

На рисунках 6–8 представлены профили трёх примитивных переменных — плотности ρ , скорости u и давления p — в три характерных момента времени: начальный ($t = 0$), промежуточный ($t = 0.2$, момент активного взаимодействия) и конечный ($t = 0.4$, после завершения взаимодействия).

В начальный момент ($t = 0$) плотность задана в соответствии с начальным профилем: сжатый газ в зоне $x \geq 0.8$ с $\rho_{sw} \approx 2.667$, область пониженной плотности $\rho_e = 0.1$ в зоне теплового следа $x \in [0.3, 0.5]$ и невозмущённый газ $\rho_0 = 1.0$ на остальной части области. В промежуточный момент ($t = 0.2$) падающая ударная волна достигла зоны теплового следа и пронизывает её: характерен сильный провал плотности в этой зоне, окруженный пиковыми значениями сжатия. Скорость распространения волны внутри разреженной каверны существенно возрастает из-за высокой локальной скорости звука. В конечный момент ($t = 0.4$) ударная волна покинула расчетную область через левую границу благодаря заданным условиям свободного выхода (Неймана).

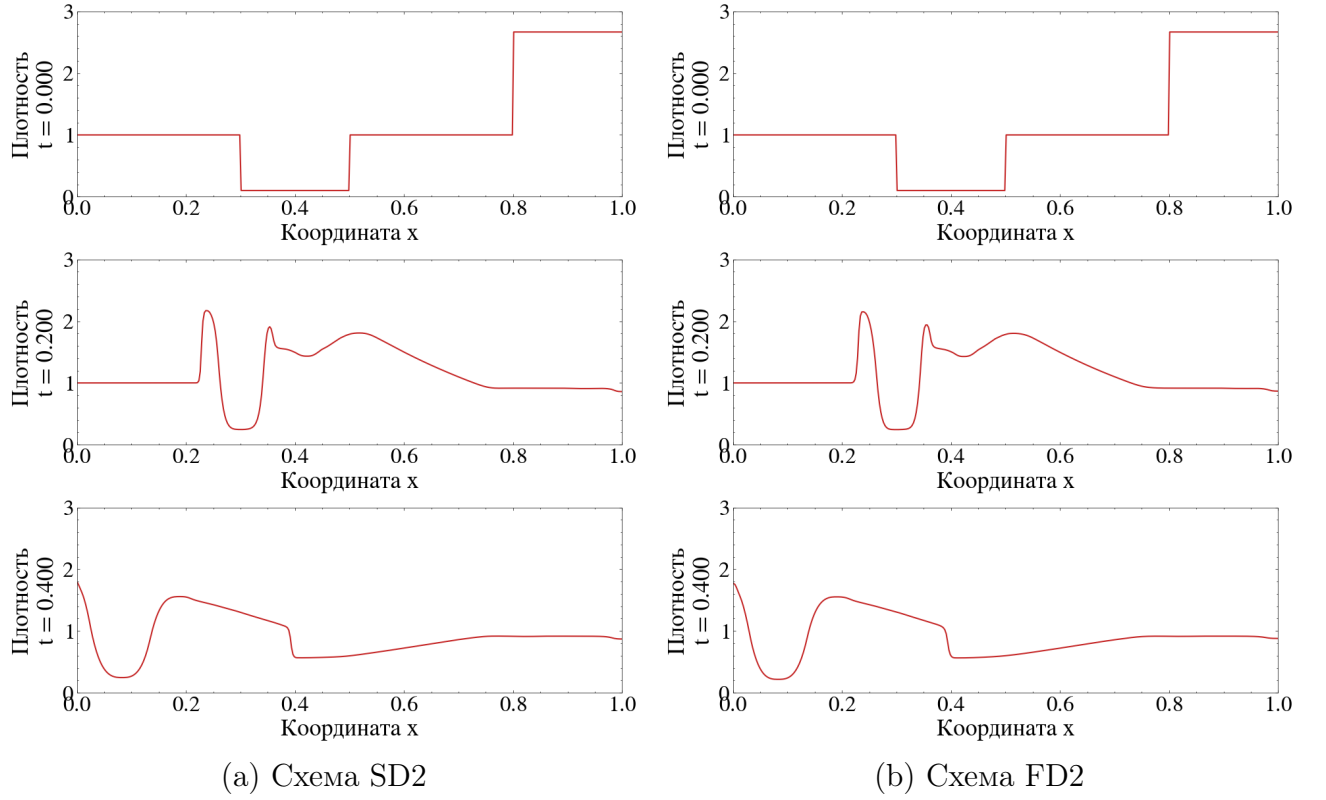
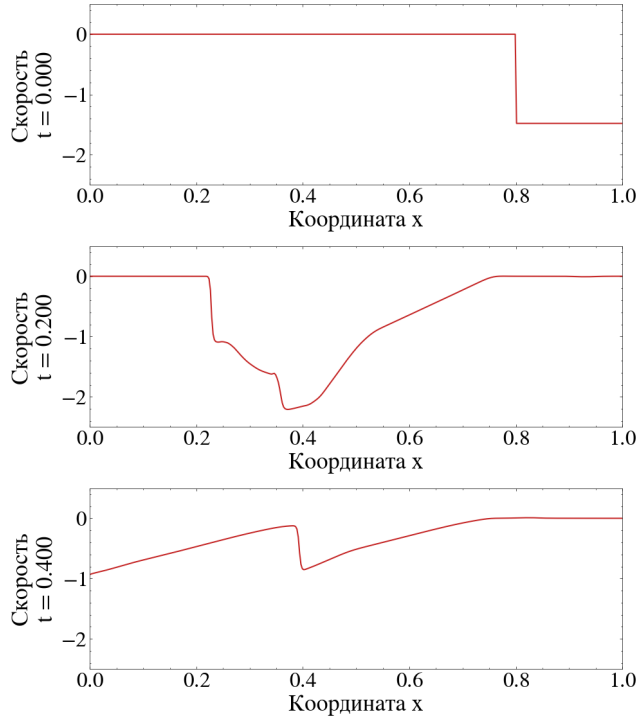


Рисунок 6. Одномерная задача. Профили плотности $\rho(x)$ в три характерных момента времени: $t = 0.0, 0.2, 0.4$. Параметры: $N = 400, CFL = 0.475$.

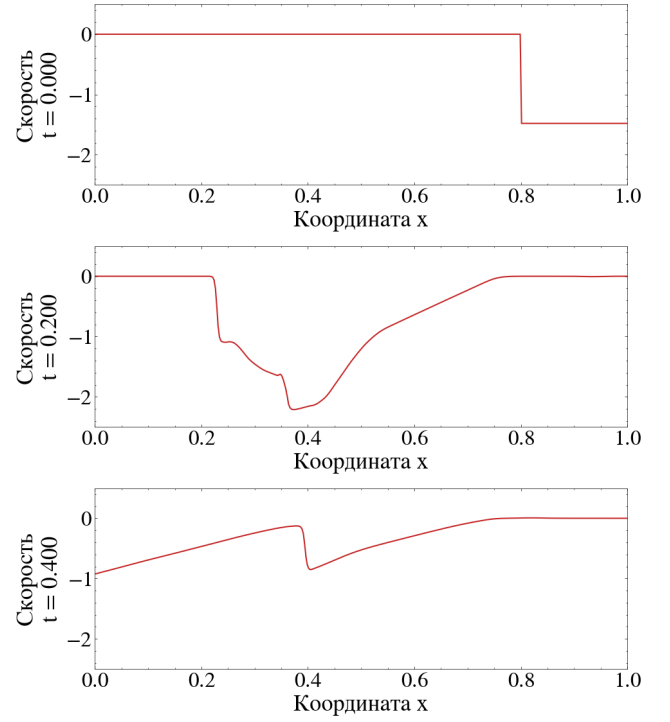
Позади неё в области теплового следа сформировалась сложная система акустических и энтропийных возмущений с характерными перепадами плотности.

Профиль скорости наглядно демонстрирует структуру волновых взаимодействий. В начальный момент газ в зоне инициации движется со скоростью $u_{sw} \approx -1.479$ в отрицательном направлении оси x . В промежуточный момент внутри каверны формируется область интенсивного течения с ярко выраженным градиентом скорости. К моменту $t = 0.4$ движение газа в целом затухает, однако профиль скорости сохраняет сложную немонотонную структуру, обусловленную множественными акустическими отражениями внутри бывшей каверны.

Давление является наиболее информативным параметром для анализа волновой структуры. Изобарное начальное условие $p_0 = 1.0$ вне ударной волны сменяется сложной картиной в момент $t = 0.2$, когда внутри разреженной зоны образуется выраженный пик давления, свидетельствующий о силь-

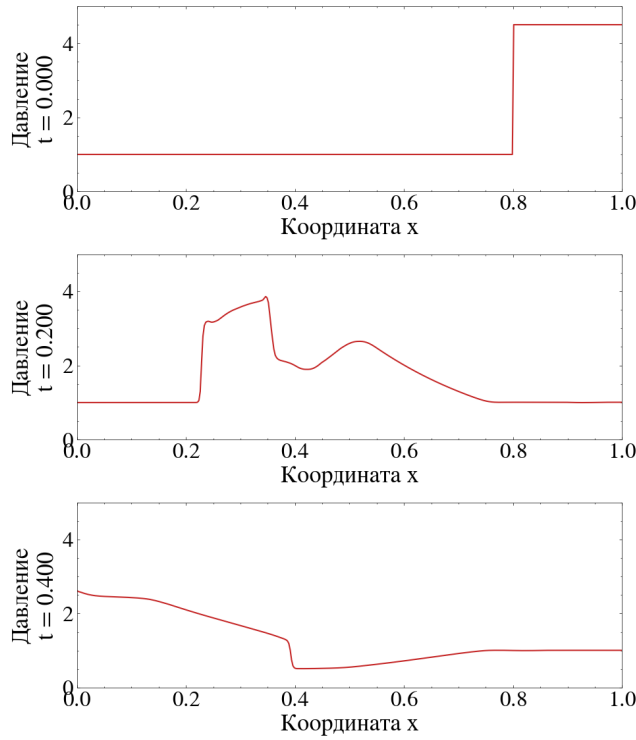


(a) Схема SD2

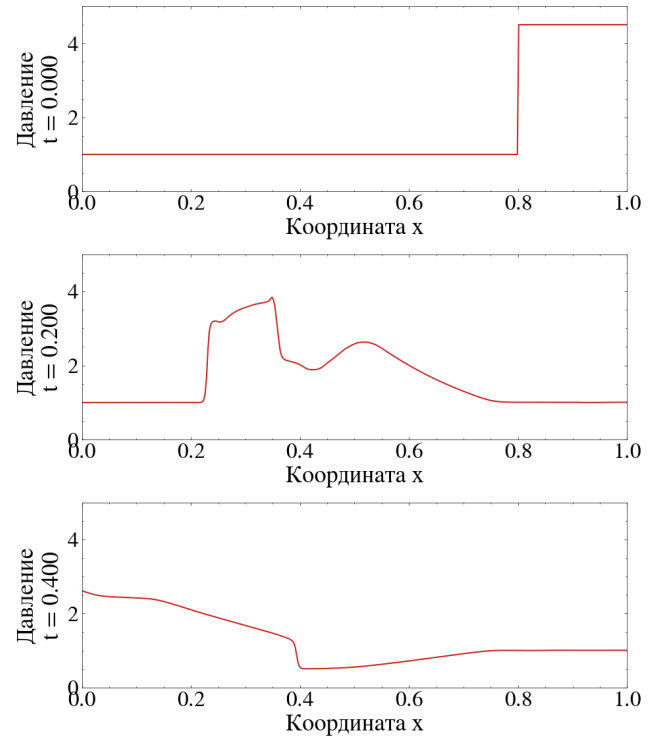


(b) Схема FD2

Рисунок 7. Одномерная задача. Профили скорости $u(x)$ в три характерных момента времени: $t = 0.0, 0.2, 0.4$. Параметры: $N = 400$, $CFL = 0.475$.



(a) Схема SD2



(b) Схема FD2

Рисунок 8. Одномерная задача. Профили давления $p(x)$ в три характерных момента времени: $t = 0.0, 0.2, 0.4$. Параметры: $N = 400$, $CFL = 0.475$.

ном сжатии газа с малой инерцией. К моменту $t = 0.4$ основное возмущение давления покидает домен, а оставшийся профиль постепенно выравнивается, стремясь к фоновому значению.

Физическая картина течения: двумерная задача

Двумерная постановка позволяет исследовать эффекты рефракции на различных геометрических формах: круглом (центр $(0.45, 0.65)$, радиус 0.08) и квадратном (ограничен координатами $x \in [0.25, 0.41]$, $y \in [0.20, 0.44]$) пузырях.

В начальный момент ($t = 0$) тепловая карта плотности воспроизводит двумерную постановку: плоский фронт у правой границы и две зоны пониженной плотности. В момент $t = 0.2$ визуализируется феномен рефракции. При прохождении плоского фронта через круглую каверну фронт дифрагирует плавно, образуя «линзу». Взаимодействие с квадратной каверной приводит к резкому искажению фронта на углах препятствия. В конечный момент ($t = 0.4$) основная ударная волна пересекает левую границу. На месте круглого пузыря формируется симметричная вихревая структура грибовидной формы: завихренность генерируется бароклинным механизмом из-за неколлинearности градиентов плотности и давления на искривлённой границе раздела [2]. На месте квадратного пузыря структура оказывается вытянутой и хаотичной.

Поле модуля скорости детально иллюстрирует кинематику процесса разрушения каверн. В промежуточный момент ($t = 0.2$) внутри обеих зон энерговложения наблюдаются ярко выраженные максимумы скорости, что связано с ускорением легкого газа под действием перепада давления на ударной волне. В круглом пузыре разгон потока происходит плавно и центростремительно. В то же время, на острых гранях квадратного пузыря возникают зоны локального торможения и отрыва потока, что приводит к формированию сложной системы сдвиговых слоев. К моменту $t = 0.4$ поле скорости визуализирует развитые вихревые течения: для круглого пузыря это пара

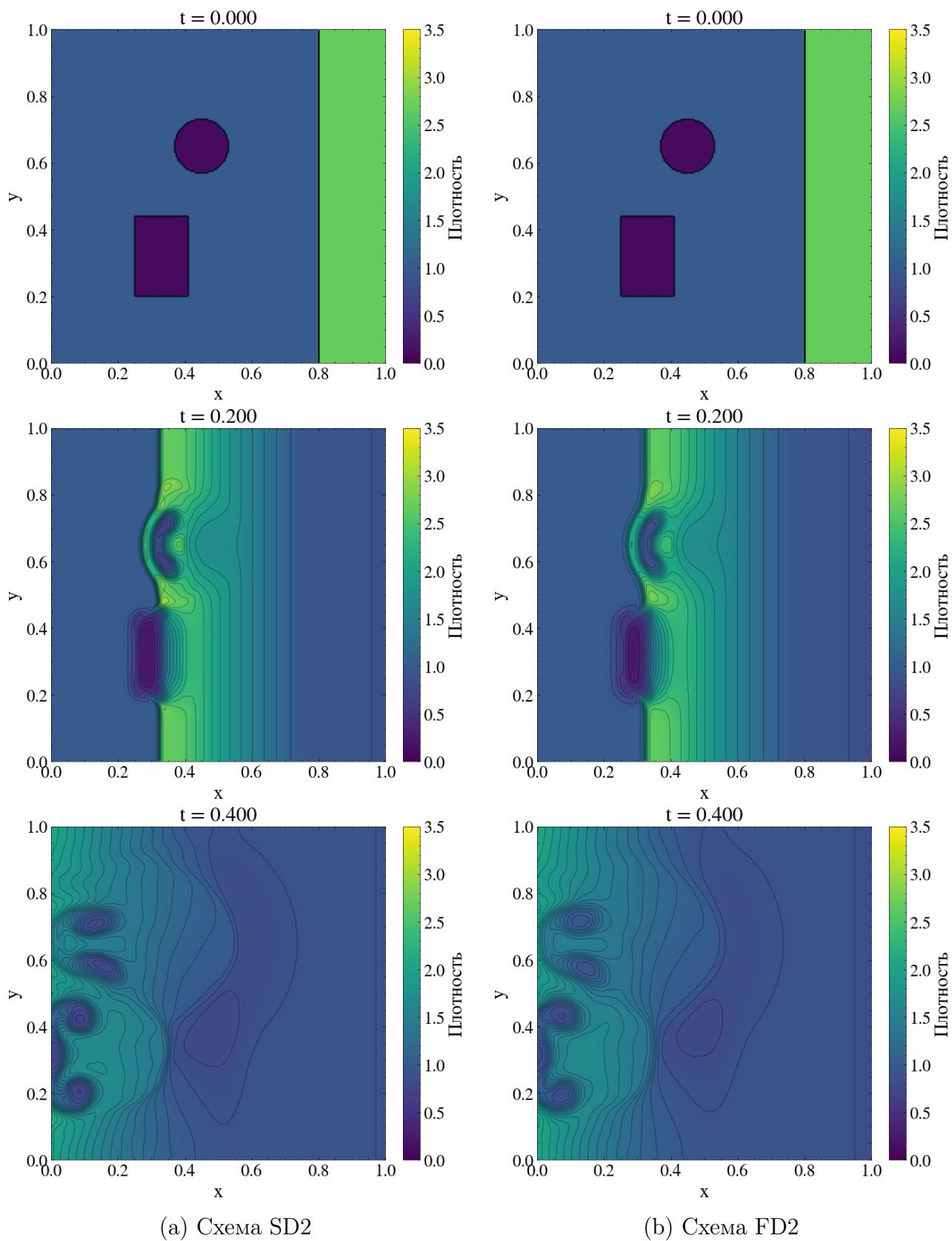


Рисунок 9. Двумерная задача. Тепловые карты плотности $\rho(x, y)$ в три характерных момента времени: $t = 0.0, 0.2, 0.4$. Параметры: Сетка 200×200 , $CFL = 0.475$.

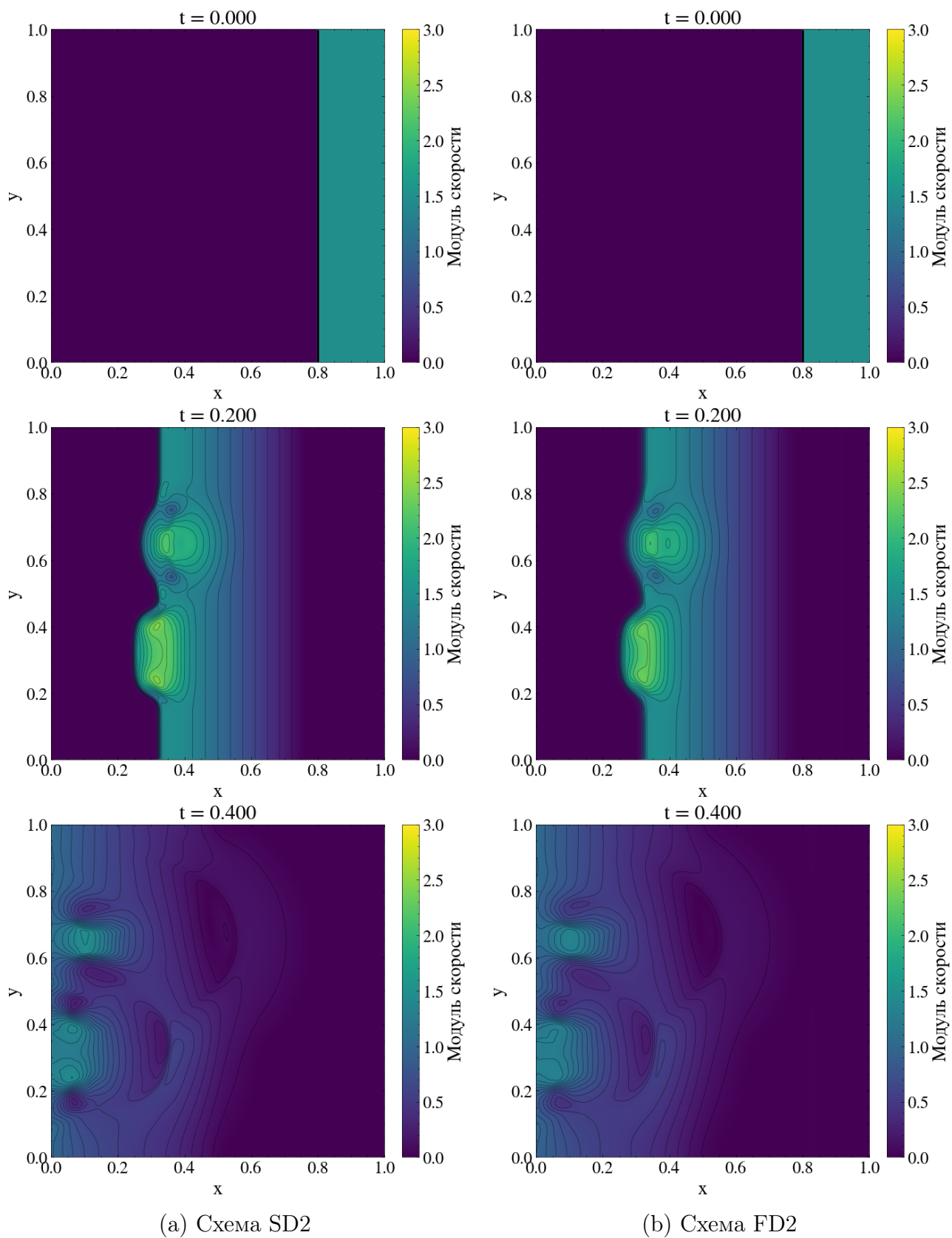


Рисунок 10. Двумерная задача. Тепловые карты модуля скорости в три характерных момента времени: $t = 0.0, 0.2, 0.4$. Параметры: Сетка 200×200 , $CFL = 0.475$.

устойчивых вихрей, для квадратного — каскад мелких вихревых структур.

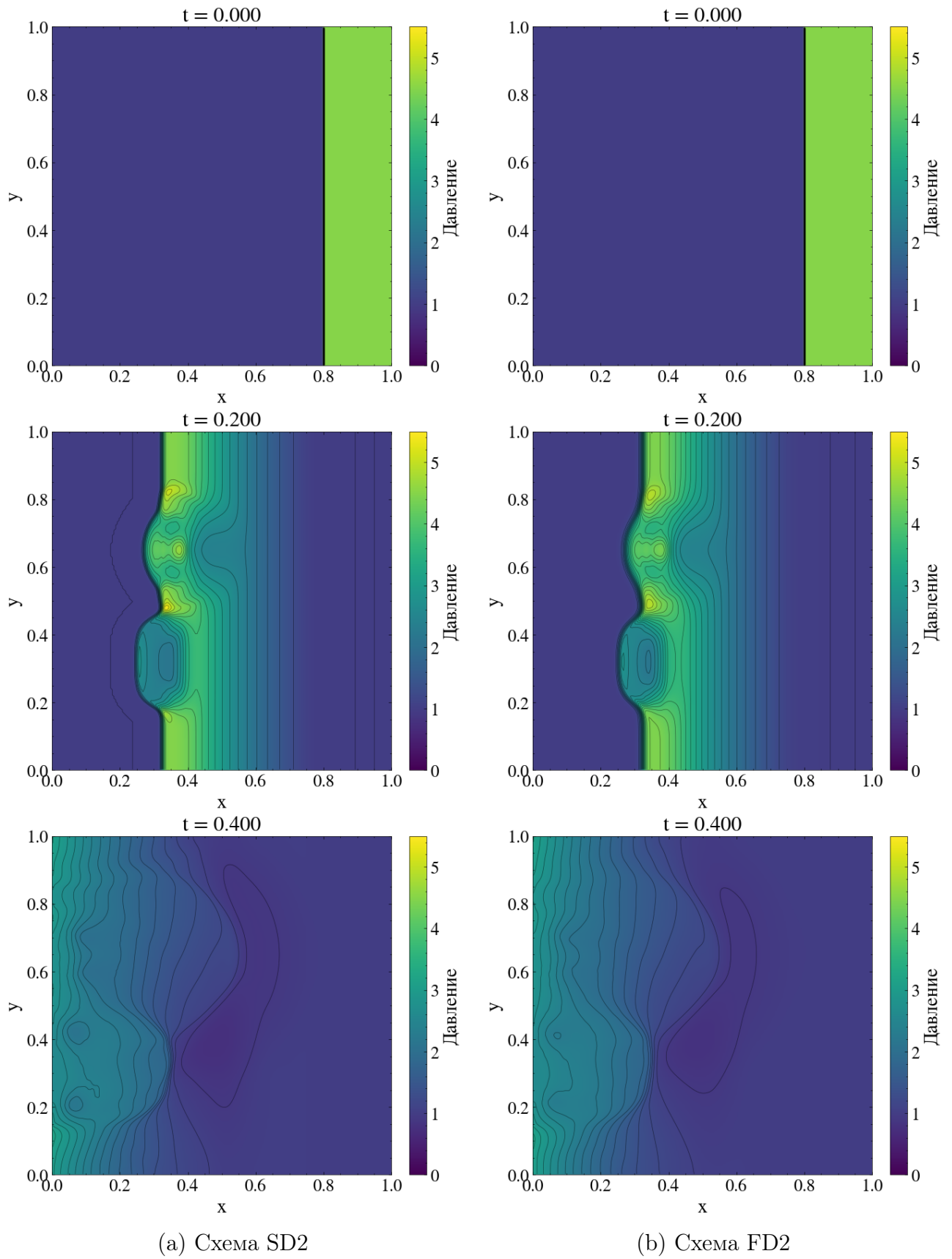


Рисунок 11. Двумерная задача. Тепловые карты давления $p(x, y)$ в три характерных момента времени: $t = 0.0, 0.2, 0.4$. Параметры: Сетка 200×200 , $CFL = 0.475$.

Тепловые карты давления подтверждают описанную акустическую динамику взаимодействия. Изобарность нарушается в момент $t = 0.2$, когда внутри легкого газа происходит интенсивное сжатие. Значительные градиенты давления локализуются на границах пузырей, причем квадратная форма порождает сильные локальные пики на углах препятствия, являющиеся источниками вторичных акустических возмущений. К конечному моменту ($t = 0.4$) макроскопическое поле давления выравнивается, оставляя лишь локальные минимумы в центрах образовавшихся вихрей, что соответствует физике вращательного движения газа.

Сравнение схем и анализ осцилляций

Сравнение результатов, полученных схемами SD2 и FD2, показывает их качественное совпадение, однако схема SD2 (на основе спектрального радиуса) демонстрирует заметно меньшую численную диссипацию. Это особенно ярко проявляется в двумерных расчётах полей плотности и модуля скорости в конечный момент времени ($t = 0.4$), где контуры мелкомасштабных вихревых структур на месте разрушенных каверн разрешаются более чётко и контрастно по сравнению со схемой FD2.

Обе схемы обеспечивают строгую монотонность решения: плотность и давление не имеют нефизичных выбросов на фронтах сильных разрывов. Успешный выход сильной ударной волны за пределы домена без генерации паразитных отражений подтверждает корректность реализации смешанных граничных условий (свободный выход слева, жесткие стенки на остальных границах), что делает разработанные алгоритмы надежным инструментом для моделирования газодинамических течений со сложной пространственно-временной структурой.

4.3. Кросс-верификация относительно JAX-FLUIDS

Для количественной проверки корректности разработанных схем SD2 и FD2 проведено сопоставление с открытой эталонной библиотекой JAX-

FLUIDS [10], использующей конфигурацию Godunov + Rusanov + MINMOD. Тестовая задача — модифицированная задача Сода [17], заданная начальными условиями (40).

Сравнение выполнено в двух характерных моментах времени:

- $t = 0.2$ — фаза взаимодействия ударной волны с контактным разрывом;
- $t = 0.4$ — пост-интерактивный устоявшийся режим.

Расчёты проведены на серии сеток $N \in \{100, 200, 400\}$ с идентичными числовыми параметрами: CFL = 0.475, ограничитель minmod, граничные условия ZEROGRADIENT на левой границе и WALL на правой. Для согласования временной дискретизации используется схема SSP-RK2 для метода SD2 и встроенный предиктор-корректор для FD2. Для FD2 предварительно устранён эффект шахматной сетки путём компенсирующего сдвига $\Delta x = -h/2$ при нечётном числе временных шагов до момента сохранения.

Качественное сопоставление профилей

На рисунках 12–14 представлены профили плотности, скорости и давления, полученные обеими схемами на сетке $N = 400$ в три выбранных момента времени. На всех временных срезах решения обеих схем визуально неотличимы от эталонных JAX-FLUIDS, включая корректное воспроизведение сложной пиковой структуры плотности и скорости в зоне $x \in [0.2, 0.5]$ на этапе взаимодействия волн ($t = 0.2$) и устоявшегося режима ($t = 0.4$).

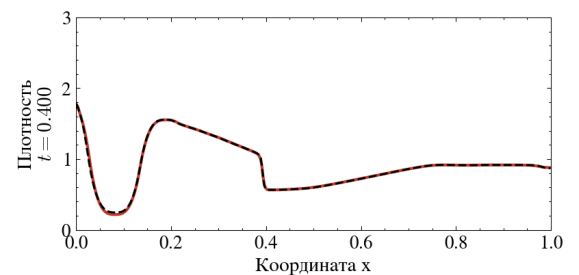
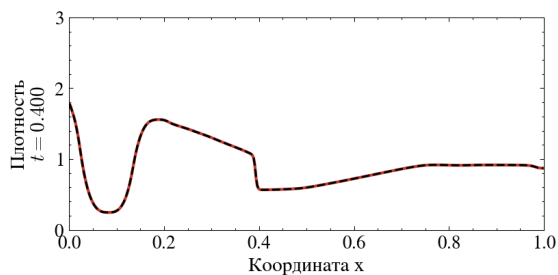
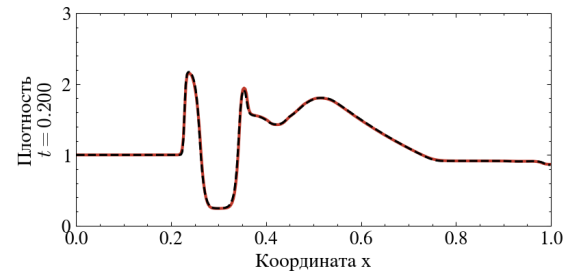
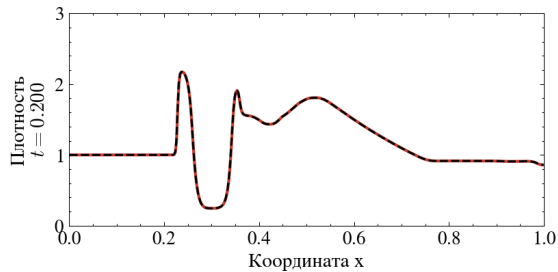
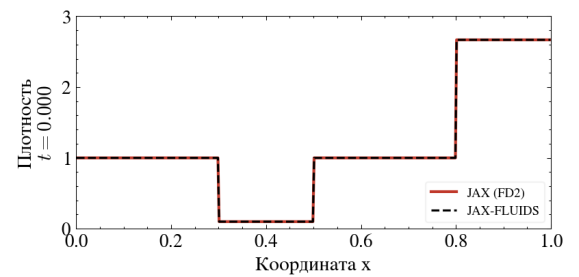
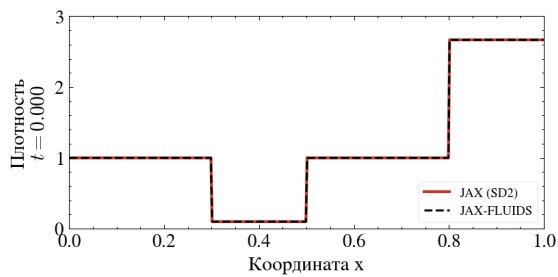
Поточечное сравнение

На рисунке 15 показана поточечная разность

$$\Delta q(x) = q_{\text{JAX}}(x) - q_{\text{JAX-FLUIDS}}(x) \quad (92)$$

полей ρ , u , p в момент $t = 0.4$ для трёх разрешений сетки.

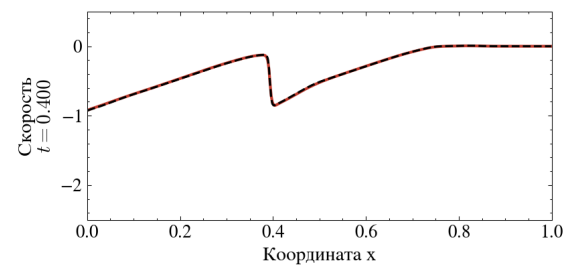
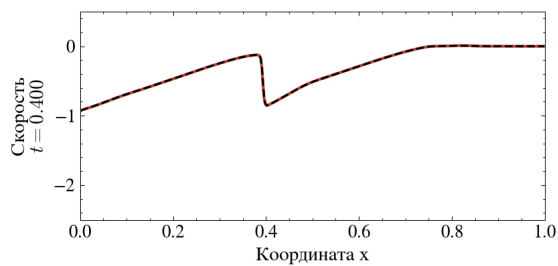
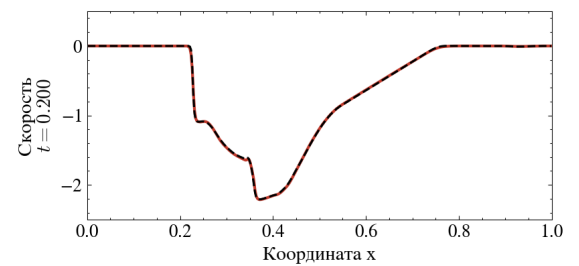
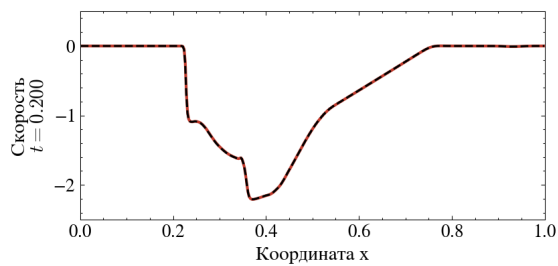
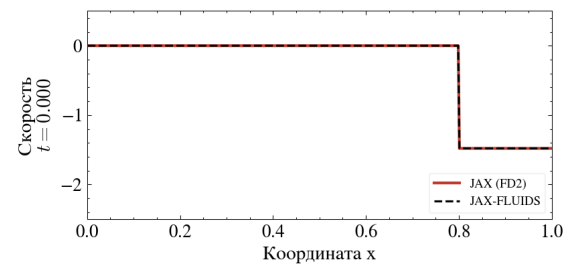
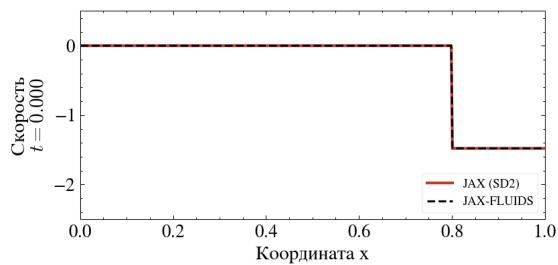
В гладких регионах решения амплитуда $|\Delta q|$ мала (порядка 10^{-2} и менее) и в целом уменьшается при измельчении сетки, тогда как на фронтах



(a) Схема SD2

(b) Схема FD2

Рисунок 12. Кросс-верификация поля плотности относительно JAX-FLUIDS



(a) Схема SD2

(b) Схема FD2

Рисунок 13. Кросс-верификация поля скорости относительно JAX-FLUIDS

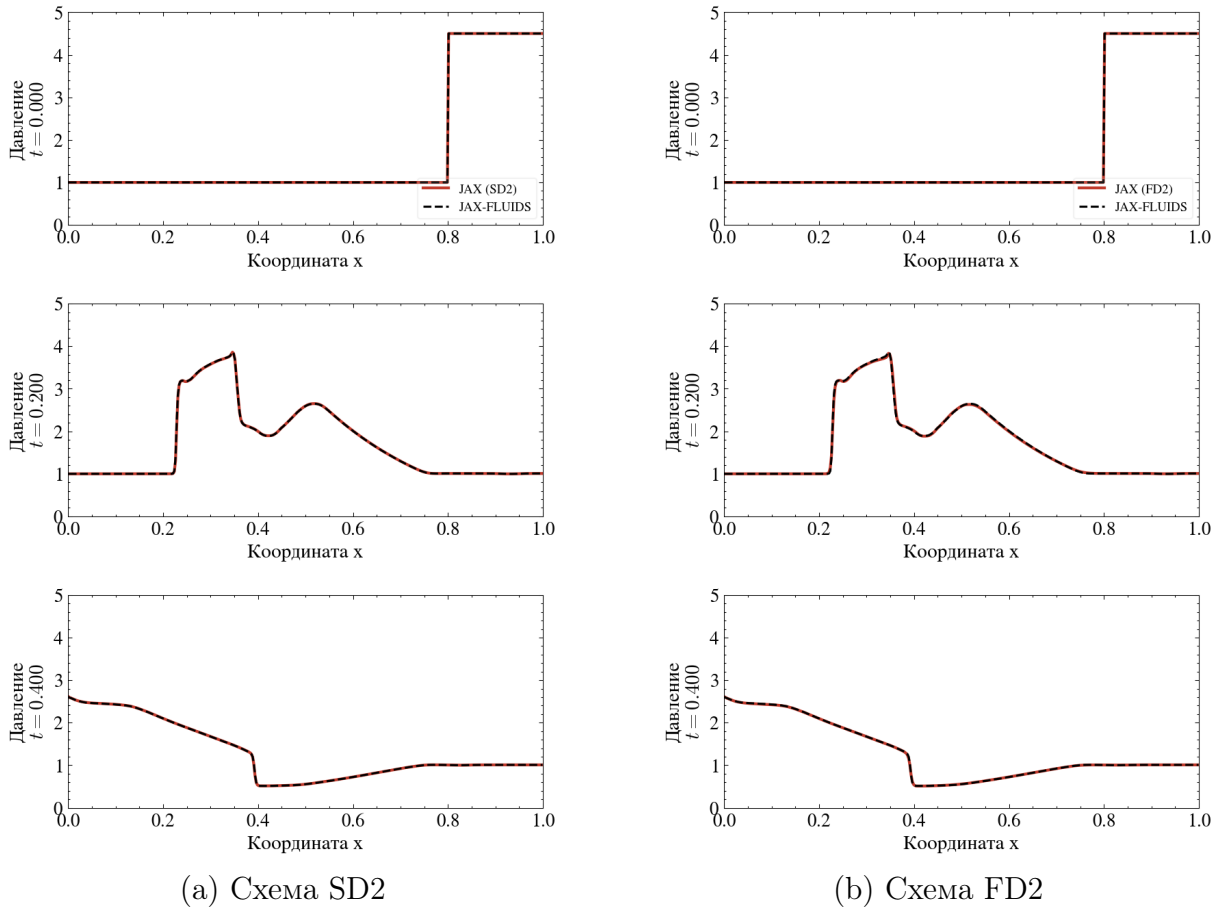


Рисунок 14. Кросс-верификация поля давления относительно JAX-FLUIDS

волн различие локализовано в области шириной $\sim h$. При этом, как поясняется ниже, убывание $|\Delta q|$ по N немонотонно, поэтому поточечная норма служит лишь иллюстративной, а не количественной мерой согласия схем.

Следует подчеркнуть, что поточечное сравнение чувствительно к фазовому сдвигу фронтов на доли ячейки, причём эта величина является немонотонной функцией разрешения, поскольку и абсолютное расхождение положений фронта $|x_A(N) - x_B(N)| \rightarrow 0$, и шаг $h_N \rightarrow 0$ стремятся к нулю с разными скоростями. Поэтому корректной мерой согласия двух схем является не норма поточечной разности, а сравнение положений фронта $x_A(N)$, $x_B(N)$ и доказательство того, что их разность убывает с измельчением сетки.

Количественное сравнение положений волновых структур

В момент времени $t = 0.4$ в решении выделены четыре локализованные волновые структуры. Положения фронтов определялись методом интерполя-

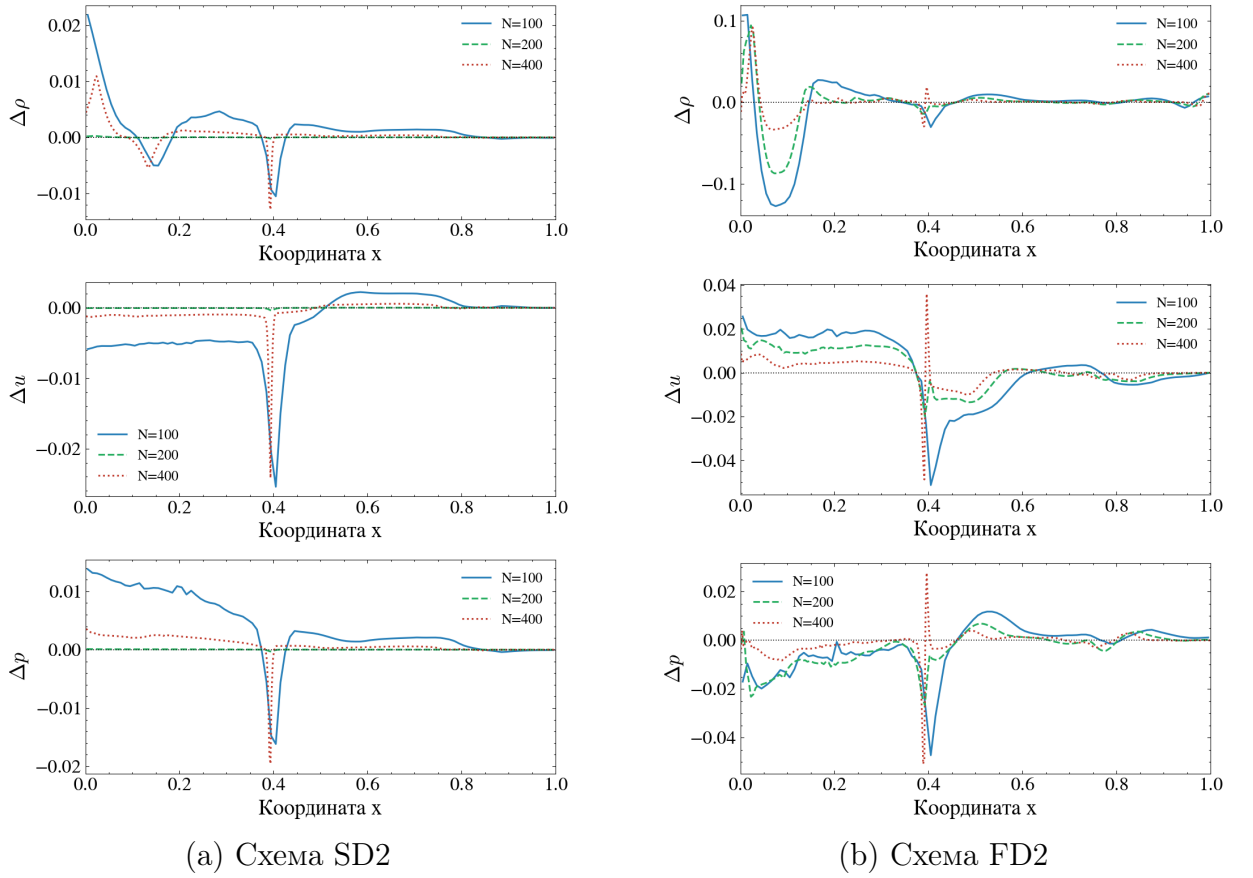


Рисунок 15. Поточечная разность профилей ρ , u , p при $t = 0.4$ относительно JAX-FLUIDS

ции пересечения уровня

$$\rho^* = \frac{\rho_{\max} + \rho_{\min}}{2} \quad (93)$$

в фиксированных пространственных окнах вокруг каждой структуры. Эта метрика устойчива к численному размытию фронта и не требует знания типа волны, а лишь наличия локализованного градиента поля.

Результаты для SD2 представлены в таблице 5.

Таблица 5. Фазовый сдвиг положений фронтов между схемой SD2 и JAX-FLUIDS в единицах ячеек $(\Delta x/h)$, $t = 0.4$.

Фронт	$N = 100$	$N = 200$	$N = 400$
Волна ≈ 0.05	+0.024	+0.001	+0.073
Волна ≈ 0.40	-0.083	-0.001	-0.072
Волна ≈ 0.60	-0.001	-0.000	-0.010
Волна ≈ 0.80	-0.082	-0.001	-0.116

Фазовый сдвиг между SD2 и JAX-FLUIDS не превышает 0.12 ячейки во

всём диапазоне исследованных разрешений. При $N = 200$ положения фронтов обеих схем совпадают с точностью $\sim 10^{-3}$ ячейки, что является следствием случайной фазовой синхронизации дискретных шагов: накопленные за ~ 160 шагов численные диссипации обеих схем выстраиваются в одну точку. Этот эффект подтверждает немонотонную природу обсуждаемой выше поточечной разности.

Экстраполяция Ричардсона первого порядка по двум наиболее тонким сеткам (таблице 6) даёт предельные положения фронтов, согласующиеся между двумя схемами с точностью не хуже 6×10^{-4} , что в четыре раза меньше шага самой тонкой использованной сетки $h_{400} = 2.5 \times 10^{-3}$. Это позволяет считать схему SD2 согласованной с эталоном в пределах их собственной численной точности.

Таблица 6. Предельные положения фронтов x^* при $h \rightarrow 0$ для схемы SD2, полученные экстраполяцией Ричардсона первого порядка по сеткам $N = 200$ и $N = 400$.

Фронт	x^* (SD2)	x^* (JAX-FLUIDS)	разность
Волна ≈ 0.05	0.027741	0.027384	3.57×10^{-4}
Волна ≈ 0.40	0.390087	0.390439	3.52×10^{-4}
Волна ≈ 0.60	0.627655	0.627706	5.10×10^{-5}
Волна ≈ 0.80	0.745819	0.746393	5.74×10^{-4}

Аналогичное сопоставление для схемы FD2 (таблица 7) демонстрирует двойственное поведение.

Таблица 7. Фазовый сдвиг положений фронтов между схемой FD2 и JAX-FLUIDS в единицах ячеек $(\Delta x/h)$, $t = 0.4$.

Фронт	$N = 100$	$N = 200$	$N = 400$
Волна ≈ 0.05	+0.232	+0.848	+1.000
Волна ≈ 0.40	-0.119	-0.067	-0.144
Волна ≈ 0.60	+0.293	+0.012	+0.277
Волна ≈ 0.80	+0.688	+1.008	+1.330

На контактном разрыве и волне разрежения согласование с эталоном составляет ≤ 0.3 ячейки, что сопоставимо с результатом для SD2. На ударных

волнах, напротив, наблюдается систематическое отклонение порядка одной ячейки, убывающее в абсолютных единицах ($|\Delta x|$ от 6.88×10^{-3} при $N = 100$ до 3.33×10^{-3} при $N = 400$) медленнее первого порядка: $|\Delta x| \propto h^{0.5}$.

Это явление имеет фундаментальную природу и обусловлено различием механизмов численной диссипации сравниваемых схем. Центральная схема Нессияху–Тадмора обладает диссипацией порядка $O(h^2/\Delta t)$, заключённой в самой структуре предиктор-корректора, тогда как Godunov + Rusanov реализует диссипацию через приближённое решение задачи Римана на гранях ячеек. Различные модифицированные уравнения двух схем приводят к слегка различным скоростям распространения ударных волн, что и проявляется как медленно сходящееся смещение их положений. Это известное теоретическое свойство центральных схем по сравнению со схемами годуновского типа описано, в частности, в работах [3, 8].

Экстраполяция Ричардсона для FD2 (таблица 8) подтверждает количественную картину: для контакта и волн разрежения предельные положения совпадают с эталоном с точностью $\sim (4\text{div}13) \times 10^{-4}$, для ударной волны — 1.61×10^{-3} . Все указанные величины меньше шага самой тонкой использованной сетки h_{400} .

Таблица 8. Предельные положения фронтов x^* при $h \rightarrow 0$ для схемы FD2, полученные экстраполяцией Ричардсона первого порядка по сеткам $N = 200$ и $N = 400$.

Фронт	x^* (FD2)	x^* (JAX-FLUIDS)	разность
Волна ≈ 0.05	0.028141	0.027384	7.57×10^{-4}
Волна ≈ 0.40	0.390051	0.390439	3.88×10^{-4}
Волна ≈ 0.60	0.629035	0.627706	1.33×10^{-3}
Волна ≈ 0.80	0.748001	0.746393	1.61×10^{-3}

Выводы по верификации

Проведённое сопоставление позволяет сформулировать ряд выводов. Обе разработанные схемы (SD2 и FD2) корректно воспроизводят волновую структуру решения, согласованную с эталоном JAX-FLUIDS на качественном

уровне во всех исследованных моментах времени, включая нелинейное взаимодействие ударной волны с контактным разрывом. Схема SD2 обеспечивает количественное согласование с эталоном на уровне ≤ 0.12 ячейки для всех волновых структур, а экстраполированные предельные положения фронтов совпадают с эталоном с точностью $\leq 6 \times 10^{-4}$, что в четыре раза меньше шага самой тонкой сетки. Схема FD2 согласуется с эталоном на контактных разрывах и волнах разрежения на том же уровне точности, что и SD2, тогда как на ударных волнах наблюдается ожидаемое теоретически расхождение порядка одной ячейки, обусловленное различием численной диссипации центральной и гудуновской схем. Наконец, экстраполяция Ричардсона подтверждает сходимость обеих схем к общему предельному решению с точностью, существенно меньшей шага самой тонкой использованной сетки. Таким образом, корректность реализаций обеих схем в одномерном случае количественно подтверждена.

Двумерная постановка позволяет проверить корректность реализаций на принципиально новых физических явлениях, отсутствующих в одномерной задаче: взаимодействии ударной волны с неоднородностями, формировании вихревых структур и развитии неустойчивости Рихтмайера–Мешкова на границах включений после прохождения шока.

На рисунке 16 представлены тепловые карты поля плотности $\rho(x, y)$, полученные тремя схемами в три характерных момента времени.

В начальный момент $t = 0$ все три схемы корректно воспроизводят заданные начальные условия с чёткими границами квадратного и кругового включений плотности $\rho = 0.1$, ровной вертикальной границей высокоплотной зоны при $x = 0.8$ и однородным фоном. Различия между схемами визуально отсутствуют, что подтверждает идентичность инициализации поля во всех трёх реализациях.

К моменту $t = 0.2$ левобегущая ударная волна от границы $x = 0.8$ достигает зоны включений и взаимодействует с ними, и все три схемы вос-

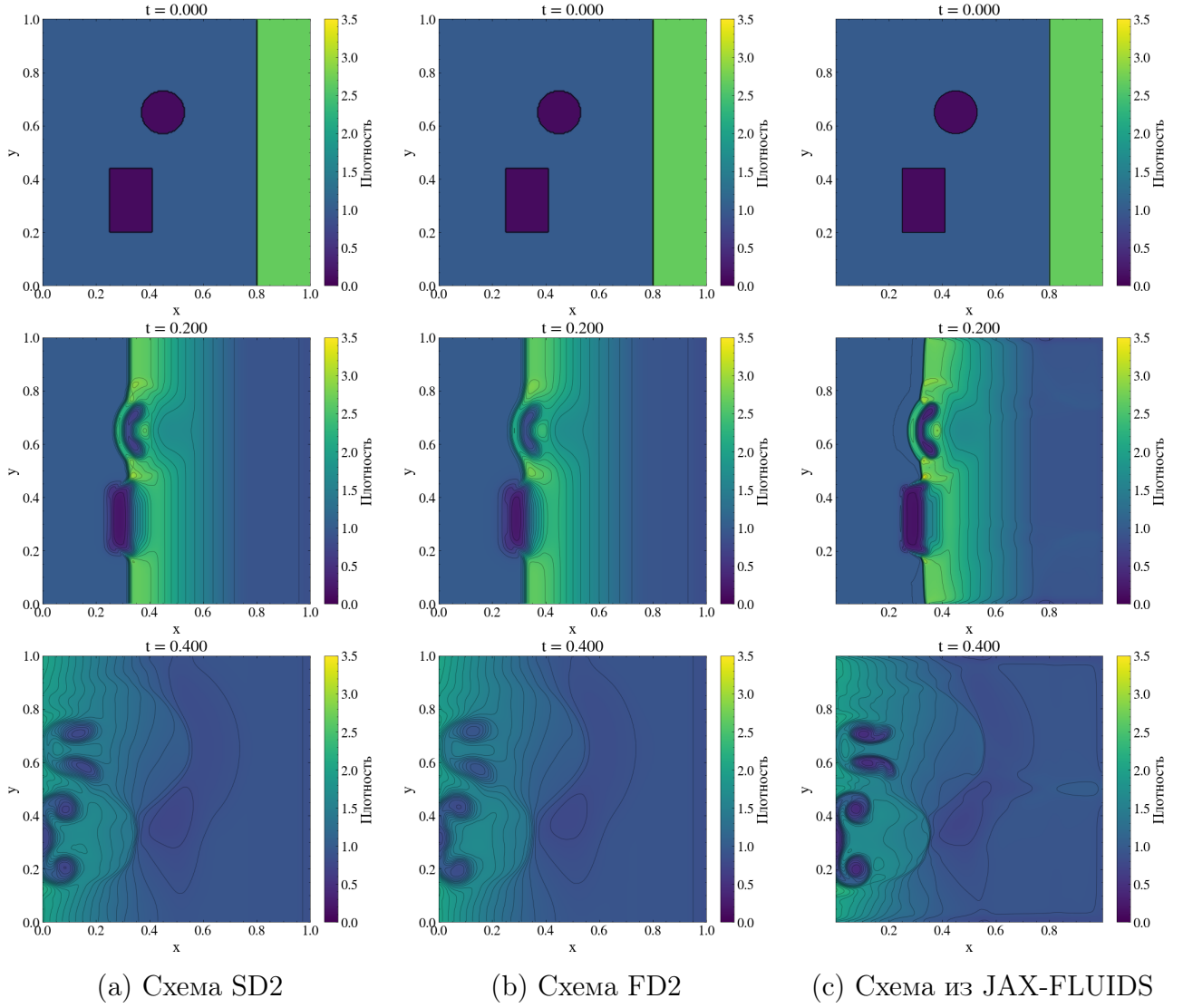


Рисунок 16. Двумерная задача. Тепловые карты плотности $\rho(x, y)$ в три характерных момента времени: $t = 0.0, 0.2, 0.4$. Параметры: сетка 200×200 , $CFL = 0.475$.

производят согласованную качественную картину. Фронт ударной волны переместился в зону $x \approx 0.35\text{--}0.40$ и сохраняет почти плоскую форму вне области включений, а за фронтом в зоне $x \in [0.4, 0.8]$ установилось однородное поле сжатой плотности $\rho \approx 2.0$. Квадратное включение деформировалось: первоначально прямоугольная область пониженной плотности сжалась горизонтально и приобрела «линзообразную» форму, отражая сжатие потоком. Круговое включение, в свою очередь, преобразовалось в характерную структуру с двумя «лопастями» — это начальная стадия формирования вихревого диполя, типичного для прохождения ударной волны через сферическое

включение. Положение фронта ударной волны и форма деформированных включений в обеих разработанных схемах визуально совпадают с эталонной картиной JAX-FLUIDS с точностью, не различимой невооружённым глазом.

К моменту $t = 0.4$ ударная волна проходит включения и продолжает движение влево, оставляя за собой развитое вихревое течение. В зоне $x \in [0, 0.2]$, $y \in [0.1, 0.8]$ во всех трёх схемах формируются вихревые структуры с характерными «глазами» пониженной плотности, возникающие в результате развития неустойчивости Рихтмайера–Мешкова на деформированных границах включений. Исходные круговая и квадратная формы оказываются полностью разрушены, на их месте образуется серия вторичных структур меньшего масштаба, а в зоне $x \in [0.4, 0.6]$, $y \in [0.4, 0.6]$ наблюдается диффузный «след» пониженной плотности, отражающий «отпечаток» прошедших через эту зону включений. Общая картина течения характеризуется градиентом плотности от значений $\rho \approx 1.5$ в левой части области к фоновому $\rho \approx 1.0$ в правой.

Крупномасштабная структура решения — число вихрей, их относительное положение и относительный размер — идентична во всех трёх схемах. Мелкомасштабные особенности, такие как точные положения и резкость экстремумов в вихревых ядрах, демонстрируют слабые количественные различия, объяснимые разной численной диссипацией схем. Схемы SD2 и JAX-FLUIDS, использующие близкие MUSCL-формулировки, дают практически идентичную мелкомасштабную картину, тогда как FD2 на основе схемы Нессияху–Тадмора показывает слегка более сглаженные мелкомасштабные структуры. Это согласуется с теоретически более высокой численной вязкостью центральной схемы по сравнению с MUSCL-схемами годуновского типа и подтверждает выводы, полученные ранее при анализе одномерной задачи.

Выводы по двумерной верификации

Качественное сопоставление двумерных расчётов показывает, что все три схемы корректно воспроизводят сложную нелинейную динамику зада-

чи, включая распространение и преломление ударной волны на неоднородностях, деформацию включений, формирование вихревого диполя и развитие неустойчивости Рихтмайера–Мешкова. Крупномасштабная структура решения — положение фронта ударной волны, число и положение вторичных вихрей — оказывается согласованной во всех трёх схемах во все исследованные моменты времени. Наблюдаемые мелкомасштабные различия между схемами не противоречат их теоретическим свойствам и не указывают на ошибки реализации, что подтверждает пригодность обеих разработанных схем (SD2 и FD2) для моделирования двумерных задач газовой динамики с разрывами и развитыми неустойчивостями. Таким образом, корректность реализаций обеих разработанных схем подтверждена и в двумерной постановке.

4.4. Анализ аппаратного ускорения

Для оценки эффективности аппаратного ускорения разработанной библиотеки было проведено сравнение её производительности с двумя открытыми решателями: классической библиотекой *centpy* [9] (реализация на NumPy, исполнение на CPU) и открытой библиотекой JAX-FLUIDS [10], также построенным на JAX/XLA. Во всех экспериментах использовалась единая физическая постановка (модифицированная задача Сода в одномерном случае и задача о взаимодействии ударной волны с двумя пузырями в двумерном), а также одинаковые параметры дискретизации: $t_{\text{end}} = 0.4$, $dt_{\text{out}} = 0.005$, $\text{CFL} = 0.475$, вычисления выполнялись с двойной точностью.

Время для решателей на основе JAX (разработанная библиотека и JAX-FLUIDS) замерялось после предварительного «прогрева» — первого запуска, в ходе которого происходит JIT-компиляция вычислительного графа средствами XLA; таким образом, в итоговый замер входит только чистое время счёта. Для *centpy*, не использующей компиляцию, фиксировалось время единственного запуска. Следует отметить, что JAX-FLUIDS реализует иную численную схему (интегратор RK2, конвективный поток годуновского типа

с приближённым решателем Римана Русанова и MINMOD-реконструкцией), тогда как разработанная библиотека использует центральные схемы SD2 и FD2. Поэтому сравнение с JAX-FLUIDS носит характер сопоставления программных реализаций в целом, а не идентичных алгоритмов.

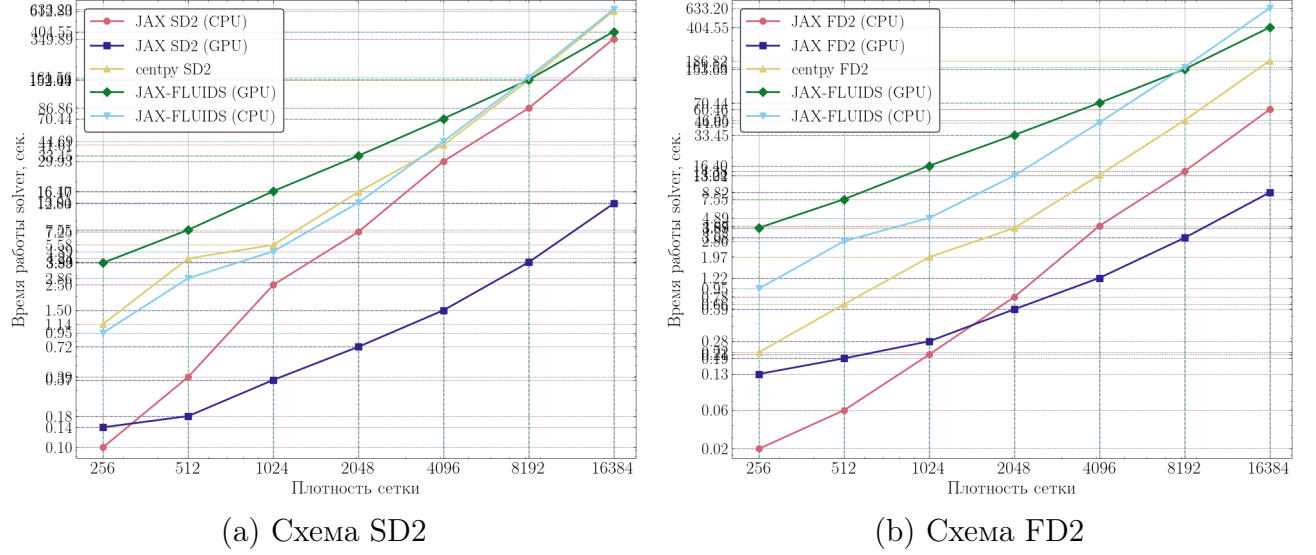


Рисунок 17. Одномерная постановка. $t_{\text{end}} = 0.4$, $dt_{\text{out}} = 0.005$, $\text{CFL} = 0.475$.

Таблица 9. Время счёта (с) для одномерной постановки, схема SD2.

J	Разраб. GPU	Разраб. CPU	<i>centpy</i>	JAX-FLUIDS GPU	JAX-FLUIDS CPU
256	0.143	0.096	1.138	3.894	0.954
512	0.179	0.394	4.244	7.545	2.857
1024	0.370	2.499	5.577	16.403	4.893
2048	0.720	7.255	16.175	33.451	13.040
4096	1.500	29.928	41.613	70.443	44.688
8192	3.931	86.862	152.440	153.630	161.556
16384	12.805	349.892	612.826	404.548	633.204

Одномерная постановка

В одномерном случае (таблицы 9, 10) ускорение от перехода на GPU оказывается умеренным, а на малых сетках GPU-версия даже проигрывает CPU-реализации (например, для схемы FD2 при $J = 256$: 0.131 с на GPU против 0.023 с на CPU). Причина в том, что одномерная задача представляет собой вектор всего из J ячеек, и при малых J графический процессор остаётся существенно недогруженным: время определяется не арифметикой,

Таблица 10. Время счёта (с) для одномерной постановки, схема FD2.

J	Разраб. GPU	Разраб. CPU	<i>centpy</i>	JAX-FLUIDS GPU	JAX-FLUIDS CPU
256	0.131	0.023	0.218	3.894	0.954
512	0.189	0.057	0.662	7.545	2.857
1024	0.280	0.207	1.972	16.403	4.893
2048	0.588	0.785	3.875	33.451	13.040
4096	1.217	4.067	13.218	70.443	44.688
8192	3.082	14.377	46.947	153.630	161.556
16384	8.819	60.462	186.817	404.548	633.204

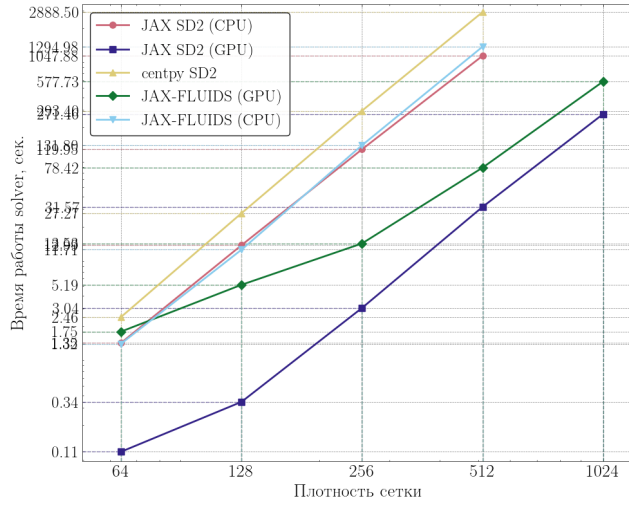
а накладными расходами на запуск вычислительных ядер и диспетчеризацию. Лишь с ростом сетки GPU выходит вперёд, достигая на $J = 16384$ ускорения $27.3\times$ (SD2) и $6.9\times$ (FD2) относительно собственной CPU-версии.

Реализации, исполняемые на CPU (*centpy* и CPU-вариант разработанной библиотеки), демонстрируют теоретический закон масштабирования $T \propto J^2$: при удвоении сетки время возрастает приблизительно в четыре раза. Так, для *centpy* (SD2) на участке $4096 \rightarrow 8192 \rightarrow 16384$ имеем $41.6 \rightarrow 152.4 \rightarrow 612.8$ с, то есть множители 3.66 и 4.02.

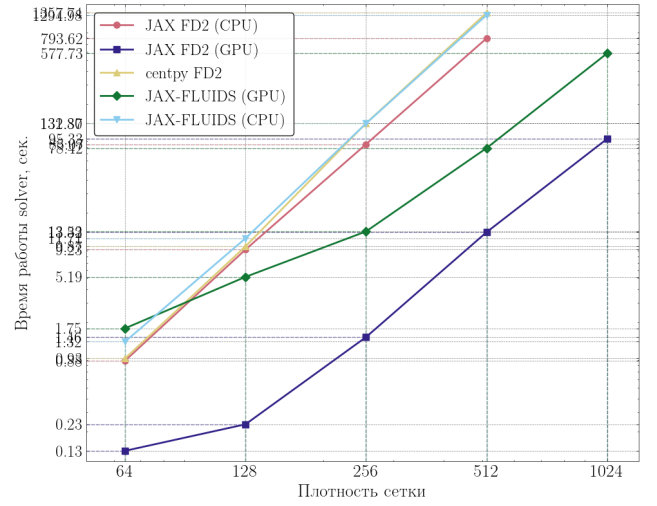
Наименьшую производительность среди решателей на JAX в одномерном случае на малых сетках показывает JAX-FLUIDS (3.89 с при $J = 256$). Это объясняется тем, что его цикл интегрирования по времени управляется на уровне Python с синхронизацией «хост–устройство» на каждом шаге, вследствие чего доминируют фиксированные накладные расходы шага. Как следствие, выигрыш от GPU у JAX-FLUIDS в 1D практически отсутствует: отношение времён CPU/GPU составляет лишь ≈ 1.6 (633.2 против 404.5 с при $J = 16384$). Разработанный решатель на GPU остаётся самым быстрым во всём диапазоне, опережая *centpy* до $47.9\times$ и JAX-FLUIDS (GPU) до $31.6\times$ (схема SD2, $J = 16384$).

Двумерная постановка

Полное преимущество GPU проявляется в двумерной постановке (таблицы 11, 12), где сетка содержит J^2 ячеек и оказывается достаточной для



(a) Схема SD2



(b) Схема FD2

Рисунок 18. Двумерная постановка. $t_{\text{end}} = 0.4$, $dt_{\text{out}} = 0.005$, $\text{CFL} = 0.475$.

Таблица 11. Время счёта (с) для двумерной постановки, схема SD2. Прочерк — расчёт не выполнялся ввиду чрезмерного времени.

$J \times J$	Разраб. GPU	Разраб. CPU	<i>centpy</i>	JAX-FLUIDS GPU	JAX-FLUIDS CPU
64	0.108	1.348	2.456	1.753	1.322
128	0.344	12.988	27.274	5.191	11.709
256	3.036	119.955	293.400	13.530	131.803
512	31.572	1047.884	2888.498	78.417	1294.979
1024	271.457	—	—	577.726	—

Таблица 12. Время счёта (с) для двумерной постановки, схема FD2. Прочерк — расчёт не выполнялся ввиду чрезмерного времени.

$J \times J$	Разраб. GPU	Разраб. CPU	<i>centpy</i>	JAX-FLUIDS GPU	JAX-FLUIDS CPU
64	0.133	0.885	0.930	1.753	1.322
128	0.232	9.233	9.865	5.191	11.709
256	1.458	85.072	132.367	13.530	131.803
512	13.316	793.620	1357.738	78.417	1294.979
1024	95.325	—	—	577.726	—

насыщения вычислительного устройства. Разработанный решатель на GPU ускоряет собственную CPU-реализацию в $33.2\times$ (SD2) и $59.6\times$ (FD2) при 512×512 и достигает сетки 1024×1024 (271 с для SD2 и 95 с для FD2), которая для CPU-реализаций и *centpy* становится практически недостижимой.

Для методов, ограниченных производительностью вычислителя, подтверждается закон $T \propto J^3$ (восьмикратный рост времени при удвоении сет-

ки): для *centpy* (SD2) на участке $128 \rightarrow 256 \rightarrow 512$ время изменяется как $27.3 \rightarrow 293.4 \rightarrow 2888.5$ с (множители 10.8 и 9.8; превышение теоретического значения 8 обусловлено эффектами иерархии кэш-памяти). Относительно *centpy* разработанный решатель на GPU оказывается быстрее на два порядка — в $91.5\times$ (SD2) и $102.0\times$ (FD2) при 512×512 .

Существенно, что в двумерном случае разработанный решатель на GPU превосходит и JAX-FLUIDS: в $2.5\times$ (SD2) и $5.9\times$ (FD2) при 512×512 , а для FD2 при 1024×1024 — в $6.1\times$ (95.3 против 577.7 с). Это объясняется тем, что универсальная открытая библиотека JAX-FLUIDS несёт большие накладные расходы по сравнению со специализированной реализацией. В отличие от одномерного случая, здесь JAX-FLUIDS уже заметно выигрывает от GPU (отношение CPU/GPU составляет ≈ 16.5 при 512×512): двумерные сетки достаточно велики, чтобы амортизировать накладные расходы Python-цикла.

Таблица 13. Коэффициенты ускорения разработанного решателя на GPU на наибольшей общей сетке ($J = 16384$ в 1D, $J \times J = 512 \times 512$ в 2D).

Сравнение	1D		2D	
	SD2	FD2	SD2	FD2
Разраб. CPU / Разраб. GPU	27.3	6.9	33.2	59.6
<i>centpy</i> / Разраб. GPU	47.9	21.2	91.5	102.0
JAX-FLUIDS (GPU) / Разраб. GPU	31.6	45.9	2.5	5.9

Влияние схемы и режимы работы GPU

Во всех экспериментах схема FD2 оказывается дешевле SD2 за счёт меньшего числа операций на ячейку (например, на GPU при 512×512 : 13.3 с для FD2 против 31.6 с для SD2). При этом относительное ускорение на GPU для FD2 даже выше, что говорит о хорошей отображаемости схемы на массивно-параллельную архитектуру.

Полученные данные отчётливо демонстрируют два режима работы GPU. На малых сетках решатель находится в режиме, ограниченном накладными расходами: время определяется числом шагов по времени ($\propto J$), а не объёмом вычислений, в результате чего при удвоении сетки время растёт лишь вдвое

(для разработанного решателя на GPU, SD2, 1D: $256 \rightarrow 512$ даёт множитель 1.25). По мере роста сетки происходит переход в режим, ограниченный производительностью вычислителя, где восстанавливается степенной закон $T \propto J^{d+1}$ (на участке $8192 \rightarrow 16384$ множитель уже 3.26, то есть близок к теоретическому 4 для 1D). Именно этим объясняется кажущаяся «недостаточность» ускорения на малых сетках: при таких размерах GPU принципиально не может быть загружен полностью.

Выводы

Проведённое сравнение показывает, что разработанная библиотека обеспечивает значительное аппаратное ускорение, в полной мере раскрывающееся в двумерных задачах. На GPU она опережает классическую реализацию *centpy* на CPU до двух порядков ($\approx 90\text{--}100\times$ в 2D) и превосходит специализированную открытую библиотеку JAX-FLUIDS в несколько раз ($2.5\text{--}6\times$ в 2D), при этом достигая разрешений сетки, недоступных для реализаций на CPU. В одномерных задачах выигрыш от GPU ограничен малым объёмом вычислений на шаг, однако и здесь разработанный решатель остаётся наиболее производительным среди рассмотренных.

ЗАКЛЮЧЕНИЕ

В рамках настоящей выпускной квалификационной работы выполнено высокопроизводительное численное моделирование распространения ударной волны в газе с локальным энерговыделением средствами разработанного программного комплекса на основе библиотеки дифференцируемого программирования JAX. Поставленная цель достигнута, а все сформулированные во введении задачи решены. Основные результаты работы состоят в следующем.

1. На основе системы уравнений Эйлера в консервативной форме сформулирована математическая модель течения идеального сжимаемого газа и поставлена модифицированная задача о прохождении ударной волны через зону локального энерговыделения (область пониженной плотности) в одномерной и двумерной постановках.
2. Выполнена программная адаптация и высокопроизводительная реализация семейства центральных разностных схем высокого разрешения SD2 и FD2 средствами библиотеки JAX. Применение JIT-компиляции, векторизации и переноса всего цикла интегрирования по времени на вычислительное устройство обеспечило исполнение единого программного кода как на центральном (CPU), так и на графическом (GPU) процессоре без его модификации.
3. Проведена верификация реализованных численных методов. На гладких задачах (синус Эйлера в одномерном случае и изоэнтропийный вихрь в двумерном) методом самосходимости на основе экстраполяции Ричардсона подтверждён заявленный второй порядок точности схем. Монотонность и неосцилляторность численного решения верифицированы на скалярных задачах с разрывами: в одномерном случае — через контроль полной вариации, в двумерном — через выполнение принципа максимума, что для схем выше первого порядка является нетривиальным результатом.
4. Выполнена кросс-верификация разработанного комплекса: результа-

ты сопоставлены с алгоритмом точного решения задачи Римана, а также с данными открытых CFD-библиотек *centpy* и JAX-FLUIDS. Профили плотности, скорости и давления показали хорошее согласие во всех рассмотренных постановках, что подтверждает корректность разработанной реализации.

5. Проведена оценка вычислительной эффективности и анализ аппаратного ускорения на CPU и GPU. Показано, что в двумерных задачах разработанный решатель на GPU ускоряет собственную CPU-реализацию до 33 раз (схема SD2) и до 60 раз (схема FD2), опережает классическую библиотеку *centpy* до двух порядков (в 90–100 раз) и превосходит специализированную библиотеку JAX-FLUIDS в 2.5–6 раз, достигая при этом разрешений сетки (1024×1024), недостижимых для реализаций на CPU за разумное время. Установлено, что наблюдаемое время счёта в режиме, ограниченном производительностью вычислителя, согласуется с теоретическими законами масштабирования $T \propto J^2$ для одномерных и $T \propto J^3$ для двумерных задач.
6. На примере модифицированной задачи с локальным энерговыделением продемонстрированы возможности разработанного комплекса: получена детальная картина динамики параметров газа — плотности, скорости и давления — при прохождении ударной волны через область пониженной плотности в одномерной и двумерной постановках.

Полученные результаты подтверждают, что применение инструментов дифференцируемого программирования и аппаратного ускорения позволяет эффективно переносить классические разностные схемы высокого разрешения на современные массивно-параллельные архитектуры, обеспечивая значительный выигрыш в производительности без потери точности. Это делает разработанный программный комплекс практически пригодным инструментом для проведения серийных ресурсоёмких расчётов задач газовой динамики.

ки на широкодоступном оборудовании, в том числе на облачных платформах.

Дальнейшее развитие работы может быть связано с обобщением комплекса на трёхмерные постановки, учётом вязких эффектов (переход к уравнениям Навье–Стокса), внедрением адаптивного измельчения сетки, а также с использованием свойства автоматического дифференцирования библиотеки JAX для решения обратных задач и задач оптимального управления параметрами зоны энерговложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Knight, D. Survey of aerodynamic drag reduction at high speed by energy deposition / D. Knight // Journal of Propulsion and Power. — 2008. — Vol. 24, no. 6. — P. 1153–1167.
2. Ландау, Л. Д. Гидродинамика. Теоретическая физика: учебное пособие / Л. Д. Ландау, Е. М. Лифшиц. — Изд. 6-е, испр. — Москва : Физматлит, 2015. — Т. 6. — 736 с.
3. Toro, E. F. Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction / E. F. Toro. — 3rd ed. — Berlin : Springer, 2009. — 724 p.
4. Куликовский, А. Г. Математические вопросы численного решения гиперболических систем уравнений / А. Г. Куликовский, Н. В. Погорелов, А. Ю. Семенов. — Москва : Физматлит, 2012. — 656 с.
5. Bradbury, J. JAX: composable transformations of Python+NumPy programs [Электронный ресурс] / J. Bradbury, R. Frostig, P. Hawkins [et al.]. — 2018. — URL: <http://github.com/google/jax> (дата обращения: 30.05.2026).
6. Hunter, J. D. Matplotlib: A 2D graphics environment / J. D. Hunter // Computing in Science & Engineering. — 2007. — Vol. 9, no. 3. — P. 90–95.
7. Nessyahu, H. Non-oscillatory central differencing for hyperbolic conservation laws / H. Nessyahu, E. Tadmor // Journal of Computational Physics. — 1990. — Vol. 87, no. 2. — P. 408–463.
8. Kurganov, A. New high-resolution central schemes for nonlinear conservation laws and convection–diffusion equations / A. Kurganov, E. Tadmor // Journal of Computational Physics. — 2000. — Vol. 160, no. 1. — P. 241–282.
9. Zenginoglu, A. centpy: Central schemes for conservation laws in Python [Электронный ресурс] / A. Zenginoglu. — 2020. — URL: <https://github.com/AnilZen/centpy> (дата обращения: 30.05.2026).
10. Bezgin, D. A. JAX-Fluids: A fully-differentiable high-order computational fluid dynamics solver for compressible two-phase flows / D. A. Bezgin, A.

- B. Buhendwa, N. A. Adams // Computer Physics Communications. — 2023. — Vol. 282. — P. 108527.
11. Годунов, С. К. Численное решение многомерных задач газовой динамики / С. К. Годунов, А. В. Забродин, М. Я. Иванов [и др.]. — Москва : Наука, 1976. — 400 с.
 12. LeVeque, R. J. Numerical Methods for Conservation Laws / R. J. LeVeque. — 2nd ed. — Basel : Birkhäuser, 1992. — 214 p.
 13. Harten, A. High resolution schemes for hyperbolic conservation laws / A. Harten // Journal of Computational Physics. — 1983. — Vol. 49, no. 3. — P. 357–393.
 14. Gottlieb, S. Total variation diminishing Runge–Kutta schemes / S. Gottlieb, C.-W. Shu // Mathematics of Computation. — 1998. — Vol. 67, no. 221. — P. 73–85.
 15. Hu, C. Weighted essentially non-oscillatory schemes on triangular meshes / C. Hu, C.-W. Shu // Journal of Computational Physics. — 1999. — Vol. 144, no. 1. — P. 210–227.
 16. Goodman, J. B. On the accuracy of stable schemes for 2D scalar conservation laws / J. B. Goodman, R. J. LeVeque // Mathematics of Computation. — 1985. — Vol. 45, no. 171. — P. 15–21.
 17. Sod, G. A. A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws / G. A. Sod // Journal of Computational Physics. — 1978. — Vol. 27, no. 1. — P. 1–31.

ПРИЛОЖЕНИЕ А

Листинг 1. Основные функции, управляющие вычислениями

```
1
def compute_dt(pars: Pars1d, eqn: Equation1d, u: jnp.ndarray
2
3     ) -> float:
4         max_speed = jnp.max(eqn.spectral_radius(u))
5         safe_speed = jnp.maximum(max_speed, 1e-6)
6         return pars.cfl * pars.dx / safe_speed
7
8 class Solver1d:
9     def __init__(self, pars: Pars1d, eqn: Equation1d,
10         scheme_name: str = "sd2", limiter_name: str = "minmod"):
11         self.pars, self.eqn = pars, eqn
12         limiter_map = {"minmod": limiters.minmod, "superbee":
13             limiters.superbee,
14             "mc": limiters.monotonized_central, "
15             van_leer": limiters.van_leer}
16         self.limiter = limiter_map.get(limiter_name,
17             limiters.minmod)
18
19         if scheme_name == "sd2":
20             self.rhs_fn = lambda t, u: schemes.
21             compute_rhs_sd2(t, u, self.pars, self.eqn, self.limiter)
22             self.step_fn, self.is_fd2 = time_integration.
23             step_ssp_rk2, False
24         elif scheme_name == "fd2":
25             self.rhs_fn, self.step_fn, self.is_fd2 = None,
26             None, True
27             self.fd2_step_jit = jax.jit(lambda u, dt, odd:
28             schemes.compute_step_fd2_1d(
29                 u, dt, self.pars, self.eqn, self.limiter,
30                 odd))
31         else: raise NotImplementedError(f"Scheme {
32             scheme_name} not implemented")
```

```

22
    self.max_snapshots = int(jnp.ceil(self.pars.t_final
23 / self.pars.dt_out)) + 2
    self.solve_jit = jax.jit(self._solve_internal)
24
25
    def _solve_internal(self, u0: jnp.ndarray):
26
        saved_states = jnp.zeros((self.max_snapshots,) + u0
27 shape)
        saved_times = jnp.zeros(self.max_snapshots)
28
        saved_states, saved_times = saved_states.at[0].set(
29 u0), saved_times.at[0].set(0.0)
30
31
        def cond_fun(state): return state[0] < self.pars.
t_final
32
33
        def body_fun(state):
34
            t, u, next_time, idx, cnt, ss, st, odd = state
35
            dt = self.pars.cfl * self.pars.dx / jnp.maximum(
jnp.max(self.eqn.spectral_radius(u)), 1e-6)
36
            dt = jnp.minimum(dt, next_time - t)
37
            dt = jnp.minimum(dt, self.pars.t_final - t)
38
            u_new = self.fd2_step_jit(u, dt, odd) if self.
is_fd2 else self.step_fn(t, u, dt, self.rhs_fn)
39
            odd_new = jnp.logical_not(odd) if self.is_fd2
40 else odd
            t_new = t + dt
41
            should_save = t_new >= next_time - 1e-9
42
            next_idx = idx + 1
43
44
            def save(args): ss, st, i = args; return ss.at[i
45 ].set(u_new), st.at[i].set(t_new)
            def no_save(args): return args[0], args[1]
46
47
            ss_new, st_new = jax.lax.cond(should_save, save,
48

```

```

        no_save, (ss, st, next_idx))

        return (t_new, u_new, jnp.where(should_save,
next_time + self.pars.dt_out, next_time),
                jnp.where(should_save, next_idx, idx),
cnt + 1, ss_new, st_new, odd_new)

        init = (0.0, u0, self.pars.dt_out, 0, 0,
saved_states, saved_times, jnp.array(False))
        (_, _, _, final_idx, total_steps, ss_final, st_final
, _) = jax.lax.while_loop(cond_fun, body_fun, init)
        return st_final, ss_final, final_idx + 1,
total_steps

def solve(self) -> Dict[str, Any]:
    dx = self.pars.dx
    x = jnp.linspace(self.pars.x_init + dx/2, self.pars
x_final - dx/2, self.pars.J)
    u0 = self.eqn.initial_data(x)
    saved_times, saved_states, actual_count, total_steps
= self.solve_jit(u0)
    actual_count_int = int(actual_count)
    return {"x": x, "t": saved_times[:actual_count_int],
            "u_n": saved_states[:actual_count_int], "
steps": total_steps}

class Solver2d:
    def __init__(self, pars: Pars2d, eqn: Equation2d,
scheme_name: str = "sd2", limiter_name: str = "minmod"):
        self.pars, self.eqn = pars, eqn
        limiter_map = {"minmod": limiters.minmod, "superbee"
: limiters.superbee,
                        "mc": limiters.monotonized_central, "
van_leer": limiters.van_leer}
        self.limiter = limiter_map.get(limiter_name,
limiters.minmod)

```

```

71
72     if scheme_name == "sd2":
73         self.rhs_fn = lambda t, u: schemes.
compute_rhs_sd2_2d(t, u, self.pars, self.eqn, self.limiter
74 )
75         self.step_fn, self.is_fd2 = time_integration.
step_ssp_rk2, False
76     elif scheme_name == "fd2":
77         self.rhs_fn, self.step_fn, self.is_fd2 = None,
None, True
78         self.fd2_step_jit = jax.jit(lambda u, dt, odd:
schemes.compute_step_fd2_2d(
79             u, dt, self.pars, self.eqn, self.limiter,
odd))
80     else: raise NotImplementedError(f"Scheme {
scheme_name} not implemented for 2D")
81
82     self.max_snapshots = int(jnp.ceil(self.pars.t_final
/ self.pars.dt_out)) + 2
83     self.solve_jit = jax.jit(self._solve_internal)
84
85     def _solve_internal(self, u0: jnp.ndarray):
86         saved_states = jnp.zeros((self.max_snapshots,) + u0
shape)
87         saved_times = jnp.zeros(self.max_snapshots)
88         saved_states, saved_times = saved_states.at[0].set(
u0), saved_times.at[0].set(0.0)
89
90     def cond_fun(state): return state[0] < self.pars.
t_final
91
92     def body_fun(state):
93         t, u, next_time, idx, cnt, ss, st, odd = state
94         sx = jnp.max(self.eqn.spectral_radius_x(u))
95         sy = jnp.max(self.eqn.spectral_radius_y(u))

```

```

        dt = self.pars.cfl / (jnp.maximum(sx,1e-6)/self.pars.dx + jnp.maximum(sy,1e-6)/self.pars.dy) 95
        dt = jnp.minimum(dt, min(next_time - t, self.pars.t_final - t)) 96

    97

    u_new = self.fd2_step_jit(u, dt, odd) if self.is_fd2 else self.step_fn(t, u, dt, self.rhs_fn) 98
    odd_new = jnp.logical_not(odd) if self.is_fd2 else odd 99

    t_new, should_save = t + dt, t + dt >= next_time - 1e-9 100

    next_idx = idx + 1 101

    102

    def save(args): ss, st, i = args; return ss.at[i].set(u_new), st.at[i].set(t_new) 103
    def no_save(args): return args[0], args[1] 104

    105

    ss_new, st_new = jax.lax.cond(should_save, save, no_save, (ss, st, next_idx)) 106

    return (t_new, u_new, jnp.where(should_save, next_time + self.pars.dt_out, next_time), 107
            jnp.where(should_save, next_idx, idx), cnt + 1, ss_new, st_new, odd_new) 108

    109

    init = (0.0, u0, self.pars.dt_out, 0, 0, saved_states, saved_times, jnp.array(False)) 110
    (_, _, _, final_idx, total_steps, ss_final, st_final, _) = jax.lax.while_loop(cond_fun, body_fun, init) 111

    return st_final, ss_final, final_idx + 1, total_steps 112

    113

    def solve(self) -> Dict[str, Any]: 114
        x = jnp.linspace(self.pars.x_init + self.pars.dx/2, self.pars.x_final - self.pars.dx/2, self.pars.Jx) 115
        y = jnp.linspace(self.pars.y_init + self.pars.dy/2, 116

```

```

self.pars.y_final - self.pars.dy/2, self.pars.Jy)
    X, Y = jnp.meshgrid(x, y, indexing='ij')
    u0 = self.eqn.initial_data(X, Y)
    saved_times, saved_states, actual_count, total_steps
= self.solve_jit(u0)
    actual_count_int = int(actual_count)
    return {"X": X, "Y": Y, "t": saved_times[:
actual_count_int],
            "u": saved_states[:actual_count_int], "steps
": total_steps}

```